On the Design of Value

A Normative Specification for Participant-Indexed Computation

Based on Contextual Compute — the General Theory of Computation

Sarva Labs, Princeton, NJ, USA
sarva.ai
March 2026

"It was never about the ordering.
It was always about the orderer."

Abstract

The Contextual Compute series establishes that value and information are categorically distinct domains [2], that the K-Machine is the correct abstract machine for value computation [3], and that the Participant–Interaction–Context (PIC) model provides the operational semantics of that machine [4]. Each of these results is semantic: they establish what value computation *means*. None of them specifies how it executes.

This paper closes that gap. We specify the normative implementation standard for any system claiming to compute over value correctly. The standard consists of four components: the *Context Protocol* (CP), which specifies how participant-indexed contexts are formed, succeeded, and composed; the *Interaction State Machine* (ISM), which specifies how linear state transformations execute; the *Interaction Finality Protocol* (IFP), which specifies how interactions achieve deterministic, local finality; and *Context Power*, which specifies how participant influence is measured as a function of accumulated interaction history rather than committed capital.

These four components are not design choices. Each is a necessary consequence of PIC's structure, derived from the six axioms established in CC4 and the K-Machine's formal definition in CC3. A system satisfying this specification computes over value correctly. A system violating any normative requirement does not—regardless of how it describes itself.

This paper is the first institutional publication in the Contextual Compute series. CC6 and CC7 (forthcoming, Sarva Labs) specify MOI as the first CC-compliant open network.

Series position: CC5 in the Contextual Compute series.
Keywords: Contextual Compute, participant-indexed computation, Interaction Finality Protocol, Context Protocol, Interaction State Machine, Context Power, linear state, normative specification, Byzantine linear agreement.

Series Positioning. CC1 [1] establishes the philosophical and quantum-theoretic foundation. CC2 [2] establishes the Domain Separation Theorem: value and information occupy categorically distinct monoidal categories. CC3 [3] introduces the K-Machine and proves the Recovery Theorem: anonymous global computation is recovered exactly at WHO = 0. CC4 [4] derives the PIC operational semantics, establishing six axioms and defining V_{PIC} as a symmetric monoidal category without a diagonal morphism. CC5 (this paper) specifies the normative implementation standard derived from those semantics. CC6 and CC7 (forthcoming) specify MOI as the first CC-compliant open network.

Contents

1 Introduction: From Semantics to Specification **6**
1.1 What PIC Establishes and What It Leaves Open . . . 6
1.2 What "Normative" Means Here . . . 6
1.3 What This Specification Is and Is Not . . . 6
1.3.1 Three Questions, Three Paradigm Shifts . . . 7
1.4 Structure of This Paper . . . 8

2 The Design Mandate **9**
2.1 The Relationship to PIC . . . 9
2.2 Constraint 1 — No Shared Mutable Value Container . . . 9
2.3 Constraint 2 — Contexts Must Be Consumed, Not Copied . . . 9
2.4 Constraint 3 — Witnesses Are Constituents, Not Validators . . . 10
2.5 Constraint 4 — Finality Must Be Local and Intrinsic . . . 10
2.6 What the Mandate Leaves Open . . . 10
2.7 Relationship to Prior Work . . . 11

3 Architectural Overview **13**
3.1 The Four Components and Their Roles . . . 13
3.2 The Derivation Map . . . 13
3.3 The Tesseract as the Unifying Record . . . 14
3.4 How the Four Components Compose . . . 14
3.5 The Scope Boundary . . . 15

4 Context Protocol (CP) **17**
4.1 The Semantic Contract . . . 17
4.2 Context Superstate: Formal Definition . . . 17
4.3 The Seven Sub-Contexts . . . 19
4.4 The Context Chain . . . 20
4.5 Primary Context and Inherited Contexts . . . 21
4.6 Context Succession . . . 23
4.7 Context Genesis . . . 24
4.8 Interaction Context Space Formation . . . 24
4.9 Context Reset . . . 25
4.10 Protocol Invariants . . . 25

4.11 Implications 26

5 Interaction State Machine (ISM) 28

5.1 The Semantic Contract 28

5.2 The State Transition Function 28

5.3 The Two-Bucket Model 29

5.4 Typed Interaction Domains 30

5.5 Structural Properties 31

5.6 Re-entrancy: The Structural Argument 31

5.7 Network Objects: The First Linear Primitive at Network Level 32

5.8 ISM vs Classical Model: A Structural Comparison 33

5.9 Compliance Requirements for ISM 33

5.10 Implications 33

5.11 The Elimination of Global Ordering 34

6 Interaction Finality Protocol (IFP) 37

6.1 The Semantic Contract 37

6.2 The Triple Achievement 37

6.3 ICS Constitution 38

6.4 The Four Phases 38

6.4.1 Phase 1 — Preparation 38

6.4.2 Phase 2 — Formation 40

6.4.3 Phase 3 — Confirmation 40

6.4.4 Phase 4 — Commit 41

6.5 The Tesseract 42

6.6 Safety and Liveness 43

6.7 Conflict Resolution Is Not Ordering 44

6.8 What IFP Specifies and What Implementing Networks Decide 44

6.9 Implications 45

7 Context Power 47

7.1 The Semantic Contract 47

7.2 Trust as Flow: The Ontological Foundation 47

7.3 The Three Components 48

7.3.1 Economic Score — Capital Commitment 48

7.3.2 Behavioral Score — Demonstrated Participation 49
7.3.3 Social Score — Identity and Network Contribution 49
7.4 Structural Properties 52
7.5 The Decay Mechanism 52
7.6 Sybil Resistance 53
7.7 Context Power and Witness Authority 54
7.8 Compliance Requirements for Context Power 55
7.9 Implications 55

8 Protocol Composition and Compliance 57
8.1 How the Four Components Compose 57
8.2 The Tesseract as the Cross-Stack Record 57
8.3 Normative Compliance Requirements 58
8.3.1 Minimum CC-Compliant Implementation 58
8.3.2 What Implementing Networks May Vary 59
8.3.3 What Implementing Networks Must Not Vary 59
8.4 The Derivation Chain 59

9 Bridge to MOI: CC Protocol Objects and Their Realisations 61
9.1 The Relationship Between CC5 and CC6/CC7 61
9.2 CC Protocol Object to MOI Realisation Map 61
9.3 What CC6 Specifies 62
9.4 What CC7 Specifies 63
9.5 The Live Reference Implementation 63

10 Closing Statement 64
10.1 What This Paper Has Established 64
10.2 The Derivation Chain Is Complete 64
10.3 What Comes Next 65
10.4 The Claim 65

A Notation and Symbol Index 67

B CC Protocol Compliance Checklist (Normative) 69

C Derivation Map 72

1 Introduction: From Semantics to Specification

1.1 What PIC Establishes and What It Leaves Open

CC4 establishes PIC as the operational semantics of the K-Machine. Its central results are precise: the participant 4-tuple $p = (\mathrm{id}, \sigma, f_p, H)$ is the irreducible computational locus; the Context Superstate

$$C(p) = \mathrm{Trust}(p) \triangleright \big\{\mathrm{Assets}(p),\ \mathrm{Storage}(p),\ \mathrm{Logic}(p),\ \mathrm{Keys}(p),\ \mathrm{Prefs}(p),\ \mathrm{Intel}(p)\big\}$$

is the semantic unit of value; the interaction 5-tuple $\iota = (S, T, W, f_\iota, \mathrm{Auth}_\iota)$ is the only mechanism by which contexts change; and the six axioms define V_{PIC} as a symmetric monoidal category in which the diagonal morphism is structurally absent.

What PIC deliberately does not establish is how any of this executes. PIC specifies that an interaction ι produces a linear transformation over joint participant contexts. It does not specify the wire format, how witnesses are selected, how attestations are collected, or how conflicts between concurrent interactions are resolved. It specifies that finality is intrinsic. It does not specify the protocol phases by which attestation is reached.

These are not oversights. PIC is an operational semantics: it specifies what the computation means, not how it executes on a physical substrate. The gap between semantics and specification is precisely what this paper fills.

1.2 What "Normative" Means Here

A normative specification does two things. It states what a compliant system must do—the minimum requirements without which CC-compliance cannot be claimed. And it states what a compliant system may vary—the degrees of freedom available to implementing networks. Both parts are essential. A specification that mandates everything is not a specification; it is a single implementation. A specification that mandates nothing provides no guarantee to participants who rely on CC-compliance as a meaningful claim.

The Contextual Compute Protocol takes the following approach. For each of its four components, it states normative requirements derived directly from PIC's axioms and the K-Machine's structure. These requirements cannot be relaxed without violating the categorical properties that CC2, CC3, and CC4 establish. It then identifies the degrees of freedom that remain—the implementation choices that are consistent with the normative requirements and that implementing networks may exercise. MOI's specific choices within those degrees of freedom are documented in CC6 and CC7.

1.3 What This Specification Is and Is Not

This is a normative document against which implementations can be evaluated for compliance. It is not a theoretical contribution (theory is complete in CC2–CC4), not an implementation guide (CC6–CC7), and not a description of MOI's engineering choices.

Four components constitute the protocol. The **Context Protocol** (CP) governs participant state: how a Context Superstate is formed at genesis, how it succeeds from one

height to the next, how contexts compose into the Interaction Context Space, and how inherited contexts are structured. The **Interaction State Machine** (ISM) governs execution: how a linear transformation over joint participant contexts is computed, what typed interaction domains are available, and what structural guarantees the execution environment must provide. The **Interaction Finality Protocol** (IFP) governs agreement: how participants and witnesses reach the attestation threshold required for a context transition to be final. **Context Power** governs participant influence: how a participant's or witness's authority within the protocol is measured as a function of interaction history, and what structural properties that measure must satisfy.

The **Tesseract** is the record produced when an IFP execution completes. It is introduced in this paper as a CC Protocol object—the crystallised four-dimensional interaction record that encodes WHAT, WHERE, HOW, and WHO as a single immutable artefact. Its storage and traversal within specific data structures belong to implementing networks and are specified in CC6.

1.3.1 Three Questions, Three Paradigm Shifts

Lamport's Paxos asked: *how can nodes agree despite failures?* Nakamoto's Bitcoin asked: *how can open networks agree without identity?* Both share an assumption never examined: all validators must agree on all operations.

The question they did not ask: *why must everyone agree on everything?*

Several recent systems parallelise when participant sets are disjoint. This is true and insufficient. Such parallelism is conditional. It holds when state is disjoint. It collapses at shared objects—and shared objects are precisely where interesting multi-party interactions occur. At that point an external sequencer is reintroduced. The trilemma is deferred, not dissolved.

This specification is a coordinator argument. Every distributed system built on Cartesian foundations assumes the coordinator must be found—either trusted, elected, assigned per slot, or determined by committed resources. The coordinator is always external to the value. This generates the trilemma.

The inversion established by Contextual Compute is not engineering. It is categorical. Value cannot exist without the participant. This is the Domain Separation Theorem stated operationally: remove WHO from a value object and the object is not incomplete—it is malformed. The participant was never absent from value computation. They were stripped out by the anonymous global machine. When you build the machine for the correct category, the participant is always present, always local, always the coordinator by ontological necessity.

Global coordination dissolves structurally—in all contexts including shared spaces. Even in a shared inherited context, each participant is sovereign over their own state. The shared context provides computation rules, not custody. There is no shared value pool for an external coordinator to sequence access to.

Every prior distributed system assumed the coordinator must be found. This specification is built on the recognition that the coordinator was never absent—because value cannot exist without them.

1.4 Structure of This Paper

Section 2 states the design mandate. Section 3 gives an architectural overview. Sections 4–7 specify each component in turn: CP, ISM, IFP, and Context Power. Section 8 states normative compliance requirements. Section 9 maps CC Protocol objects to their MOI realisations. Section 10 closes the derivation chain.

2 The Design Mandate

2.1 The Relationship to PIC

PIC's six axioms are necessary consequences of the K-Machine's structure. They are not design choices—each is forced by a specific property of participant-indexed linear computation, and none can be removed without collapsing the model back into classical information processing. Any protocol that implements PIC must therefore satisfy all six axioms. The design mandate of CC Protocol is the translation of those axioms into implementation requirements: the minimum set of things a protocol must do to be the thing PIC says it must be.

Axiom 7.1 (Conservation) requires that any protocol correctly account for value across all participating contexts. Axiom 7.2 (Consent) requires that no context transition occur without cryptographic authorisation from its owner. Axiom 7.3 (Linearity) requires that no context be duplicated, forked, or partially consumed—and that this be structural, not merely enforced. Axiom 7.4 (Witness Sufficiency) requires that validity be defined by attestation threshold, not by global agreement. Axiom 7.5 (Composability) requires that interaction outputs serve as valid inputs to subsequent interactions without additional coordination. Axiom 7.6 (Commutativity) requires that interactions over disjoint participant sets be categorically independent.

These six axioms collapse into four implementation constraints that every CC-compliant protocol must satisfy.

2.2 Constraint 1 — No Shared Mutable Value Container

In a Cartesian system, value can be placed in a container that exists independently of any participant: a row in a shared database, an account balance in a replicated ledger, a mapping in a shared state machine. Any participant with the appropriate access can read or modify it. The container is the primary object; participants are secondary — external agents that act on state that pre-exists and outlasts them. The Linearity axiom prohibits this at the protocol level. Value must reside in exactly one participant's context at any given height. There is no valid notion of a value object that exists independently of a participant—a value object $v = \langle p, c, x \rangle$ that loses p or c is not a value object; it is anonymous content.

A CC-compliant protocol must therefore provide a state model in which value objects are participant-indexed by construction, not by convention. It is not sufficient for an application layer to enforce that assets are participant-owned if the underlying state model admits the contrary. The diagonal morphism must be absent from the substrate, not suppressed by policy.

2.3 Constraint 2 — Contexts Must Be Consumed, Not Copied

The Linearity axiom further requires that when a context transitions from height h to height $h{+}1$, the pre-context at height h ceases to exist. There is no valid context $C(p)$ at two distinct heights simultaneously. This eliminates double-spend by making the pre-

condition of any second interaction structurally unavailable—not by detecting and rejecting it, but by ensuring the pre-context no longer exists to be referenced.

A CC-compliant protocol must therefore implement context succession as a consumption-and-production operation: $C(p)$ at height h is consumed, $C(p)$ at height $h{+}1$ is produced, and no operation can reference the height-h state after the transition. The context chain formalises this:

$$\alpha(h+1) = H\big(\mathrm{id}(p) \parallel (h{+}1) \parallel \alpha(h) \parallel \sigma(h{+}1)\big)$$

where each anchor commits to its predecessor, making the succession both immutable and verifiable in $O(h)$ hash operations.

2.4 Constraint 3 — Witnesses Are Constituents, Not Validators

The Witness Sufficiency axiom establishes that agreement is constituted by participant witnessing, not discovered through external validation. The distinction is categorical. A validator is an observer of replicated state who participates in a second phase of agreement after execution. A witness is a constituent of the value interaction itself—part of the entanglement that defines the value object. Adding or removing a witness changes the interaction. A different witness set implies a different value object.

A CC-compliant protocol must therefore embed witness sets in participant state—they must be part of $\mathrm{Trust}(p)(p)$, committed in the context chain, not assigned at runtime from an external pool. The protocol must define validity in terms of the witness threshold $2f{+}1$ of $3f{+}1$, and it must ensure that validity is constituted by that attestation, not confirmed by a subsequent global process.

2.5 Constraint 4 — Finality Must Be Local and Intrinsic

The Commutativity axiom establishes that interactions over disjoint participant sets are categorically independent. The Linearity axiom establishes that context consumption is irreversible. Together they imply that finality—the property of a completed interaction—must be local to the interaction's constituent set and intrinsic to the interaction's completion. Global finality is undefined in V_{PIC} because global value is undefined. An interaction between Alice and Bob achieves finality when its constituent set attests to the completion. No other participants are involved, and no global confirmation is required.

A CC-compliant protocol must therefore achieve finality within the Interaction Context Space—the composite context formed by the named participants and their witness sets—without appealing to any global ordering mechanism, global state, or external confirmation. Finality in 2Δ (two network delay bounds after GST) follows as a consequence of this constraint, not as a performance target.

2.6 What the Mandate Leaves Open

The four constraints above are necessary. They are not sufficient to determine a unique protocol. Many protocol designs satisfy all four constraints. The Contextual Compute Protocol specifies one such design—the one implemented by MOI. Two questions the mandate leaves open are particularly significant.

First, how witnesses are selected. The mandate requires that witness sets be embedded in Trust$(p)(p)$ and that validity require $2f+1$ attestations. It does not specify how the $3f+1$ witnesses are chosen, how they rotate, or how additional witnesses beyond the bound set are incorporated. These are choices available to implementing networks, documented for MOI in CC6.

Second, how execution is expressed. The mandate requires that interactions be linear transformations over joint participant contexts. It does not specify the programming language or execution environment in which those transformations are written. The ISM defines typed interaction domains as a normative structure; the specific programming model used to extend those domains is an implementing network choice.

2.7 Relationship to Prior Work

The Contextual Compute Protocol inherits from three established bodies of work, extends two, and departs from two in ways that are categorical rather than incremental.

Inherited: Byzantine fault-tolerant consensus. The safety and liveness analysis of IFP is situated within the partial synchrony model established by Dwork, Lynch, and Stockmeyer [6] and the Byzantine fault tolerance framework of Lamport, Shostak, and Pease [19]. The $3f+1$ constituent threshold, the GST assumption, and the 2Δ finality bound are direct consequences of this framework. Castro and Liskov's PBFT [7] demonstrated practical BFT agreement; IFP inherits the core safety argument and departs on scope—PBFT operates over global replicated state, IFP operates over a predetermined Interaction Context Space. The broader FT computing tradition achieves reliability through state replication, which requires the diagonal morphism on the replicated objects. CC Protocol achieves fault tolerance through witness attestation and cryptographic commitment instead: witnesses hold hashes and signatures—Cartesian objects that may be freely copied—not copies of value objects, which are linear and exist in exactly one participant context at exactly one height. Reliability and linearity are simultaneously achievable because the fault tolerance mechanism operates entirely within the information bucket and is structurally absent from the value bucket.

Inherited: Linear type theory and categorical foundations. The two-bucket model and the Linearity axiom are grounded in Girard's linear logic [11] and its computational interpretations by Wadler [18] and Abramsky [12]. The categorical argument—that value objects inhabit a symmetric monoidal category without a diagonal morphism, while information objects inhabit a Cartesian category with one—is standard monoidal category theory. Fox's theorem [14] establishes that the diagonal morphism characterises Cartesian structure; CC2 applies this to the separation of value from information. The K-Machine and PIC model (CC3, CC4) extend this categorical foundation to computation with an explicit participant dimension.

Inherited: Self-sovereign identity. The W3C Decentralised Identifier specification [24] and Verifiable Credentials framework [25] establish the foundational primitives for participant-controlled identity. Trust$(p)(p)$ and the Social score's identity sub-component build on these standards. The CC Protocol's departure is structural embedding: rather than identity as an application-layer credential presented to a system, Trust$(p)(p)$ makes identity a constituent of the participant's own Context Superstate, committed in their context chain and verifiable without a central registry.

Extended: Object-ownership models. The Move language [23] and its deployment in object-centric networks introduced the insight that owned objects can be processed in parallel without consensus, because disjoint ownership implies disjoint state. The CC Protocol extends this in two directions. First, by making the participant—not the object—the computational primitive: participant-indexed state is the substrate, not a labelling convention applied to anonymous objects. Second, by resolving the shared-object problem structurally: in object-ownership models, shared objects require global consensus because they remain anonymous state in a global space. In the CC Protocol, what appears to be a shared space is a participant M with its own sovereign $C(M)$, its own witness set embedded in $\mathrm{Trust}(p)(M)$, and local IFP finality. The ordering authority for interactions touching $C(M)$ is M's witness set, not the full network.

Extended: Proof of Stake mechanism design. Proof of Stake consensus [22] correctly identifies that committed capital can provide Sybil resistance and align validators with network health. Context Power inherits this insight and extends it: capital commitment is the Economic score E_e^v, one of three components. The Behavioral score B_e^v and Social score S_e^v address dimensions that capital alone cannot capture—demonstrated participation, verified identity, and geographic distribution. The structural consequence is a non-conserved, non-transferable influence measure that resists capital concentration as a mathematical property rather than a governance aspiration.

Departed from: Global state machine replication. The Ethereum execution model [21] and its successors treat the network as a single replicated state machine: $\sigma' = \Upsilon(\sigma, T)$. Every transaction takes the entire world state as input and produces the entire world state as output. The CC Protocol departs from this at the substrate level. The ISM transition function $\delta(C(p_1), \ldots, C(p_n), \iota) = (C'(p_1), \ldots, C'(p_n))$ takes only the named participants' contexts as input. The scope of every interaction is determined before execution begins by the participant set, not discovered at runtime from global state. This is not a parallelisation strategy applied to a global state machine. It is a different substrate.

Departed from: Probabilistic finality and global ordering. Nakamoto consensus [10] achieves agreement through probabilistic accumulation—finality is a function of confirmation depth, not a threshold event. Leader-based BFT protocols such as Tendermint [20] and HotStuff [8] achieve deterministic finality but over a global validator set with a coordinator per slot. IFP departs from both: finality is deterministic and local, constituted at the moment the constituent set $C_\iota = P_\iota \cup W_\iota$ satisfies the commit rule. No global validator set is involved. No coordinator is required. The scope of agreement is the ICS, and the ICS is participant-constituted before the first protocol message is sent.

3 Architectural Overview

3.1 The Four Components and Their Roles

The Contextual Compute Protocol consists of four components. Each realises a distinct semantic object from PIC. None is optional—removing any one leaves the protocol unable to satisfy one or more of the four constraints in Section 2.

The **Context Protocol** (CP) realises the participant and context semantics of PIC. It specifies how a Context Superstate $C(p)$ is structured, how it is formed at genesis, how it succeeds from height h to height $h{+}1$, and how multiple contexts compose into the Interaction Context Space (ICS) that an interaction operates over. CP is the substrate on which every other component operates. ISM reads and writes contexts. IFP finalises context transitions. Context Power measures context history. Without CP there is no participant-indexed state for any of them to act on.

The **Interaction State Machine** (ISM) realises the interaction semantics of PIC. It specifies how the linear transformation f_ι in the interaction 5-tuple $\iota = (S, T, W, f_\iota, \mathrm{Auth}_\iota)$ is expressed and executed. ISM defines typed interaction domains—protocol-recognised categories of transformation with well-defined semantics and deterministic state transitions—alongside a general-purpose execution layer for transformations that do not reduce to a standard domain primitive. ISM is the computation engine of the protocol.

The **Interaction Finality Protocol** (IFP) realises the witness semantics of PIC. It specifies how the constituent set $C_\iota = P_\iota \cup W_\iota$ reaches the attestation threshold required for a context transition to be final. IFP operates over the ICS formed by CP and executes the transformation specified by ISM. Its output is the Tesseract—the crystallised interaction record that is the protocol's unit of finality.

Context Power realises the trust accumulation semantics of PIC. Specifically, it realises the formal correspondence between $\mathrm{EPI}(P, t) = g_p(t)$ from CC3's geometric model and the operational measure of participant influence within the protocol. Context Power specifies how participant influence is computed as a function of interaction history, what structural properties that function must satisfy, and how it governs witness authority. It is the protocol's answer to the question CC3 identifies as the K-Machine's fourth structural consequence: trust as flow, not stock.

3.2 The Derivation Map

Every component is a realisation, not a design choice. Table 4 makes the derivation explicit.

Table 1: K-Machine derivation chain: CC3 geometric → CC4 semantic → CC5 protocol

CC3 geometric	**CC4 semantic**	**CC5 protocol**
Manifold M — participant value space	$p = (\mathrm{id}, \sigma, f_p, H)$	CP — genesis, succession, ICS
Tangent space T_pM, KCC trajectory τ_p	$C(p)$ at height h	Network Objects — $C(p)$ at successive heights
Geodesic $\gamma : [0,1] \to M$	$\iota = (S, T, W, f_\iota, \mathrm{Auth}_\iota)$	ISM — $\delta(C(p_1), \ldots) = (C(p_1)', \ldots)$
Context singularity f_{pq}	Intrinsic finality; Value = Agreement	IFP — four phases, Tesseract
Metric tensor $g_p(t)$, $\mathrm{EPI}(P, t)$	$\mathrm{Trust}(p)$ — witnesses, $\mathrm{Intel}(p)$ anchored	Context Power — $E + B + S$, decay

3.3 The Tesseract as the Unifying Record

linear transformation over the composite context—the result is crystallised as a Tesseract. The Tesseract is named for the four dimensions of the K-Machine: WHAT, WHERE, HOW, and WHO.

A Tesseract $\mathcal{T}(\iota)(\iota)$ is produced exactly once per completed interaction. It is *bilateral*—it records the updated contexts of all participants in P_ι simultaneously. It is *final*—once produced, the pre-contexts it consumed no longer exist; there is no frame within which the interaction can be revised. It is the interactive edge between participant context chains—the cross-reference that links Alice's context history to Bob's at the moment their interaction resolved. It also creates the transitive links within each participant's context chain to record the transformation.

The Tesseract is a CC Protocol object. Its definition—what it contains and what properties it satisfies—belongs here. How Tesseracts are stored, linked, and traversed within a specific data structure belongs to implementing networks and is specified in CC6.

3.4 How the Four Components Compose

The four components operate in a fixed sequence for any interaction.

CP provides the participant contexts: it is the source of $C(p)$ for each $p \in P_\iota$ at their current heights, maintained through genesis, succession, and reset. The Interaction Context Space $C_\iota = \bigoplus_{p \in P_\iota} C(p)$, together with the union of their witness sets, is formed as the first step of interaction setup—before execution begins. No runtime discovery is required; every element of the ICS is already present in participant state.

ISM executes the transformation: given the ICS, it applies the typed domain instruction or general-purpose logic that f_ι specifies, computing the post-contexts $C'(p)$ for each $p \in P_\iota$. ISM execution is deterministic—given the same ICS and the same f_ι, it produces the same outputs.

IFP finalises the transition: it collects witness attestations over ISM's output until the

$2f+1$ threshold is reached, at which point the pre-contexts are consumed, the post-contexts are produced, and the Tesseract is crystallised. This is finality.

Context Power governs who may serve as a witness throughout. It determines selection probability and voting authority within IFP, and it accumulates as a consequence of completed Tesseracts—each finality event updates the Context Power of every participant involved.

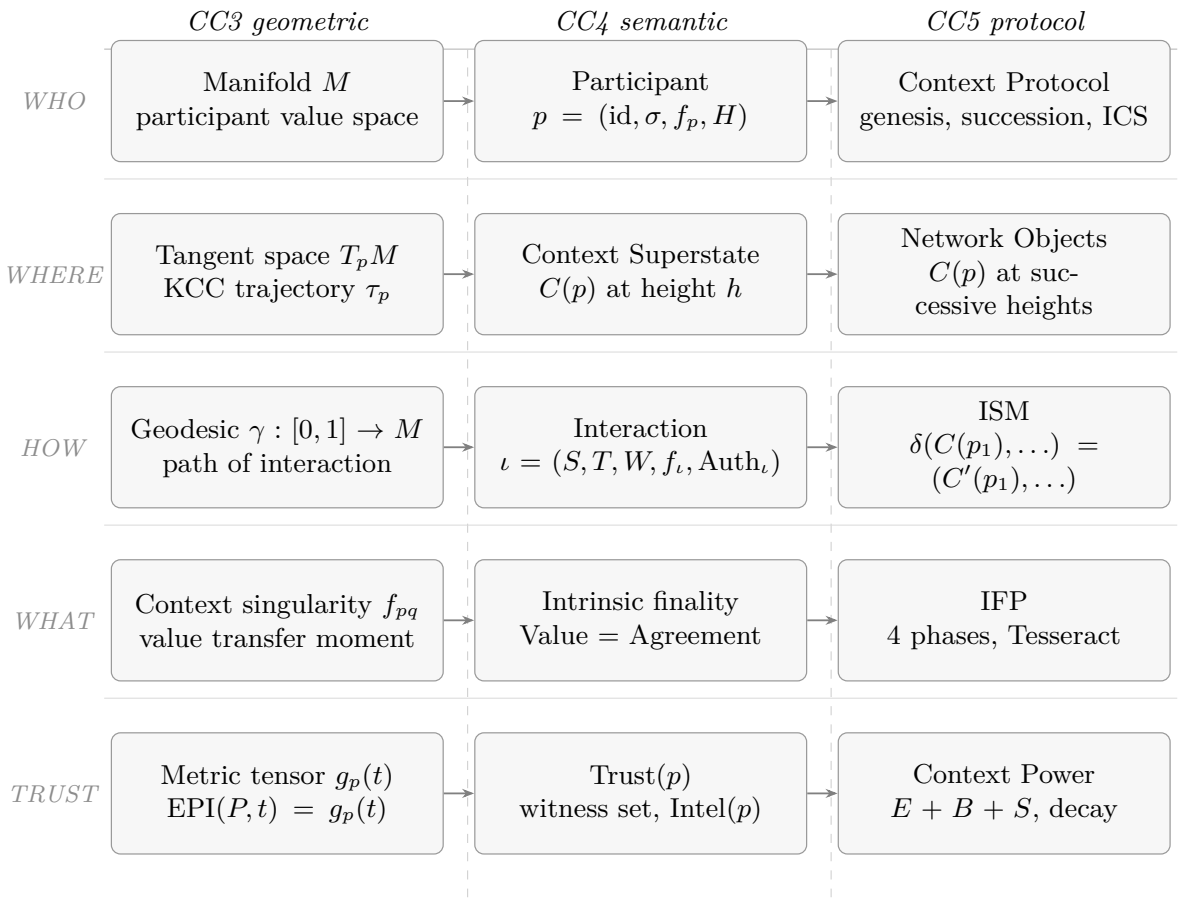

Figure 1: K-Machine derivation chain: CC3 geometric → CC4 semantic → CC5 protocol. Each row corresponds to one K-Machine dimension. Every CC5 protocol component is a realisation of a specific CC3 geometric object via its CC4 semantic counterpart.

3.5 The Scope Boundary

The Contextual Compute Protocol specifies the abstract semantics and normative requirements for each component. It does not specify:

- The concrete data structure in which contexts are stored and linked across heights. This is the MDAG in MOI—specified in CC6.
- The mechanism by which witnesses are selected beyond the bound witness set embedded in $\mathrm{Trust}(p)(p)$. VRF-based stochastic witness selection is a MOI choice—specified in CC6.

- The programming language in which ISM logic is expressed. COCO is MOI's implementation—specified in CC6.
- The specific economic parameters of Context Power: decay constants, component weights, bootstrap schedules. These are MOI parameters—specified in CC6 and CC7.

The scope boundary is the correct architectural separation between what must be true of any CC-compliant system and what MOI has chosen within that space.

4 Context Protocol (CP)

4.1 The Semantic Contract

The Context Protocol realises the participant and context semantics established in CC4. PIC defines the participant 4-tuple $p = (\mathrm{id}, \sigma, f_p, H)$ and the Context Superstate $C(p) = \mathrm{Trust}(p)(p) \triangleright \{\mathrm{Assets}, \mathrm{Storage}, \mathrm{Logic}, \mathrm{Keys}, \mathrm{Prefs}, \mathrm{Intel}\}$ as semantic objects. CP specifies how those objects are instantiated, maintained, and composed on a live network. Every requirement in CP derives from PIC's axioms. Every implementation choice that CP leaves open is a genuine degree of freedom available to implementing networks.

The foundational inversion CP embodies is this: traditional distributed systems ask "what data exists and who can access it?" CP inverts the question to "who is involved and what does their involvement mean?" The answer to the second question is context. A participant in a CC-compliant system is not an account, an address, or an identity credential attached to a system. The participant is the system at their locus. Context is not data about a participant—it is the complete representation of a participant's computational existence within the network.

4.2 Context Superstate: Formal Definition

The Context Superstate $C(p)$ for participant p is the complete, cryptographically committed representation of p's computational existence at a given height. It is a composite structure anchored by $\mathrm{Trust}(p)(p)$, with six remaining sub-contexts deriving their participant-indexed character from their entanglement with $\mathrm{Trust}(p)(p)$.

Definition 4.1 (Context Superstate)**.** The Context Superstate for participant p at height h is:

$$C(p) = \mathrm{Trust}(p)(p) \triangleright \big\{\mathrm{Assets}(p),\ \mathrm{Storage}(p),\ \mathrm{Logic}(p),\ \mathrm{Keys}(p),\ \mathrm{Prefs}(p),\ \mathrm{Intel}(p)\big\}$$

where $\triangleright$ denotes structural anchoring—not hierarchical containment. $\mathrm{Trust}(p)(p)$ does not contain the other six sub-contexts; it anchors them. Without Trust anchoring, the six sub-contexts are anonymous records. With Trust anchoring, they are participant-indexed sovereign state.

Two properties distinguish $C(p)$ from any state representation in existing distributed systems.

Completeness. $C(p)$ is the single authoritative cryptographic root of the participant's entire computational existence—a Merkle commitment to all seven sub-contexts. Content may be distributed across nodes for availability; what is singular and final is the commitment itself.

In today's distributed systems, a participant's identifier points at records held in a globally replicated state machine—a row in a database, an entry in a wallet, a profile in an app. These records exist independently of the participant and can be read, copied, and reasoned about without any involvement from them. They are attributes of logic—answers to WHAT and HOW—not independent existence associated with the participant. The WHO is absent.

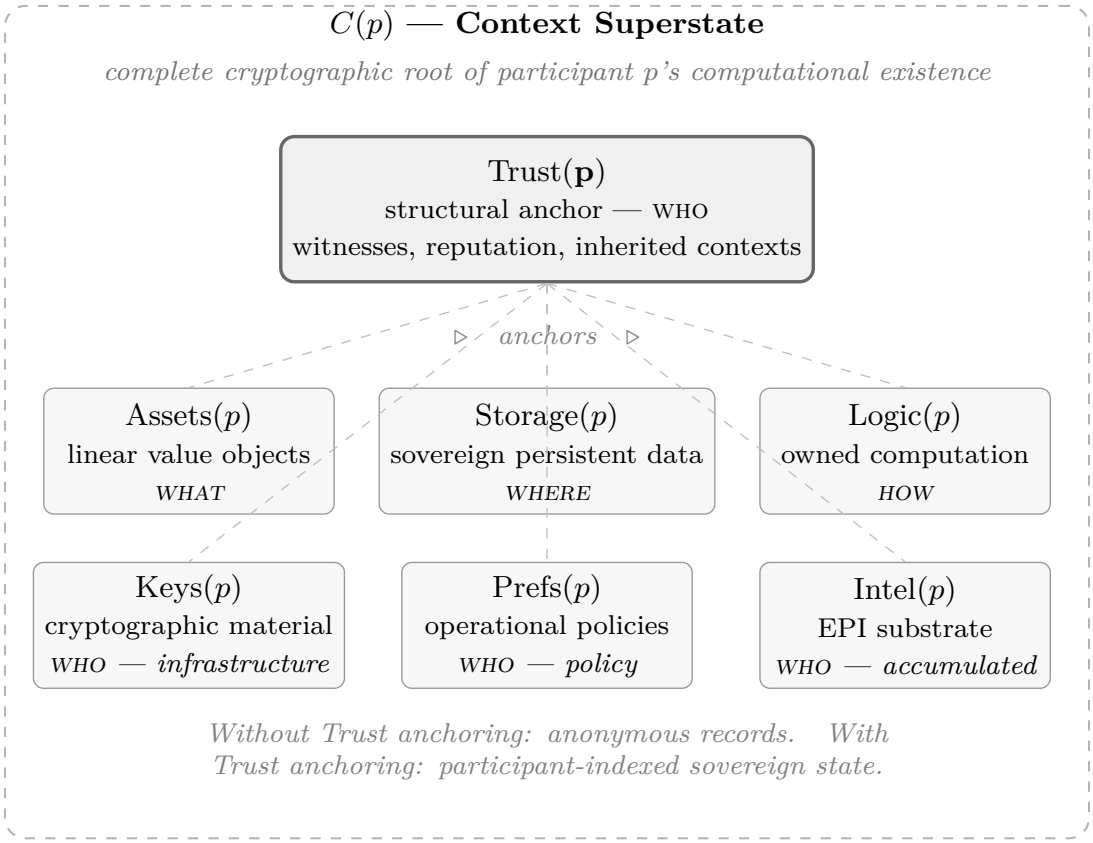

Figure 2: Context Superstate $C(p)$: Trust(p) structurally anchors six sub-contexts across the K-Machine dimensions. Trust(p) does not contain the sub-contexts—it anchors them.

$C(p)$ is different in kind: it is the participant's own sovereign anchor, computable only from their interaction history and verifiable against that history in $O(h)$ hash operations.

Deterministic knowability. There exists exactly one valid $C(p)$ per participant per height—and this fact is knowable deterministically, without external consensus. On a Cartesian substrate, the question "what is Sally's current balance?" initiates a permanent recursion: to know the current state you need consensus; to run consensus you need to know the current state. The circle never closes. Sally's balance is not a local fact—it is an emergent property of a global coordination process that cannot fully terminate. It may be *controlled* by process, but it is never *determined* by structure. There could be three copies of Sally's state or three thousand; the protocol manages which copy wins, but cannot eliminate the copies, because copying is what the substrate does.

In CP, the question "what is Sally's agreed balance?" has a deterministic answer. $C(p)$ at height h is anchored by $\alpha(h)$, attested by $W(p)$, and committed in a chain that any party can verify locally. There is no external process required to establish which state is authoritative.

The distinction is not probabilistic. After sufficient confirmations, the probability of reversion in a well-designed Cartesian system approaches zero. The probabilistic critique alone is insufficient—it sounds like a performance complaint, and Cartesian systems have a practical answer to it. The deeper issue is where finality is anchored.

In Cartesian systems, finality is non-local: indifferent to participant and context. Every

interaction, regardless of who is involved or what trust requirements the participants have specified, must pass through the same global ordering mechanism. This is not a design flaw. It is the necessary consequence of anonymous state: state with no structural owner can only achieve finality through a process that involves the whole network. A coffee payment and a property transfer queue behind each other with the same latency profile, the same trust model, the same committee—because the protocol has no structural way to distinguish them.

This is why even the fastest globally consistent systems cannot resolve this. Google Spanner uses atomic clocks to bound global consistency to milliseconds—the line moves extraordinarily fast. But it is still one line. The problem is not the speed of the line. It is that there is only one.

The deeper structural point concerns ordering. Traditional distributed systems extend partial orderings to total orderings. In CP, the natural partial ordering is preserved. Events affecting disjoint participant contexts are not ordered relative to each other—not because ordering is forbidden, but because there is no state shared between them that ordering would protect. The absence of ordering is the correct structure for a system in which state is participant-indexed by construction.

There is a deeper conflation in classical distributed systems worth naming precisely. Reliability and copyability are treated as the same axis: if you want fault tolerance, you replicate. The assumption is so embedded in Cartesian substrate design that it is rarely stated. But they are orthogonal dimensions. Reliability means the system continues to function despite node failures. Copyability means data can be duplicated across nodes. On a Cartesian substrate the only mechanism for reliability is replication—you copy state to multiple nodes so that if one fails, others have it. But this is a substrate constraint, not a logical necessity. In participant-indexed computation, reliability comes from witness attestation: $2f+1$ of $3f+1$ witnesses attest to the state transition. The state is not copied; it is witnessed. Fault tolerance is achieved through entanglement, not replication. You can be linear and fault-tolerant simultaneously. These are not competing properties; they operate on different axes entirely.

The argument applies with equal force in shared computational spaces. In a shared inherited context inhabited by multiple participants simultaneously, each participant brings their sovereign state. The shared context provides computation rules, not custody. There is no shared pool. There is no shared object for an external coordinator to sequence access to.

> *This is not a performance improvement over consensus. It is the elimination of the condition that makes external consensus necessary. What remains is linear agreement—scoped exactly to the participants and witnesses of that context, and nothing beyond them.*

4.3 The Seven Sub-Contexts

CP derives exactly seven sub-contexts from the K-Machine's four dimensions (WHAT, WHERE, HOW, WHO). The correspondence is direct and forced—each sub-context realises a dimension of the K-Machine's value space $\mathcal{H}_K = \mathcal{H}_x \otimes \mathcal{H}_y \otimes \mathcal{H}_z \otimes \mathcal{H}_p$.

Assets(p)**:** Linear value objects—tokens, balances (spendable, delegated, locked, mandated), asset metadata. The WHAT dimension. Assets exist in exactly one participant context at exactly one height. This is linearity made concrete: no copy operation exists at the protocol level.

Storage(p)**:** Participant-owned persistent data—key-value state, documents, credentials, blob references. The WHERE dimension. Storage is sovereign: no operation outside an interaction involving p can read or modify Storage(p).

Logic(p)**:** Participant-owned computation—interaction handlers, execution rules, versioned code, immutable when sealed. The HOW dimension. Logic governs how p participates in interactions.

Keys(p)**:** Cryptographic material—active key pairs, rotation history, recovery paths, revocation records, delegation weights, signature algorithm specifications including classical and post-quantum schemes. The infrastructure layer of the WHO dimension.

Prefs(p)**:** Operational policies—compute locus preferences, storage locus preferences, access policies, deny lists, privacy preferences. The policy layer of the WHO dimension.

Intel(p)**:** EPI substrate—participant-specific accumulated intelligence, anchored to p's interaction history. EPI (Emergent Participant Intelligence), established in CC3, is the formal measure of how a participant's computational influence evolves continuously as a function of their interaction history—the operational form of the metric tensor $g_p(t)$ on the participant manifold. The richer and more consistent a participant's interaction history, the higher their EPI, and therefore the greater their influence within the contexts they inhabit. Non-transferable and anchored to Trust(p)(p). The accumulated intelligence layer of the WHO dimension.

Trust(p)**:** Witness anchoring—the witness set $W(p)$ statefully embedded; inherited contexts (roles and shared spaces); Trust Quotient; Reputation Index; network behaviour attestations; delegation authorities and mandate chains. The t-dimension made concrete. Trust(p)(p) is the structural anchor of $C(p)$: the six remaining sub-contexts derive their participant-indexed character from their entanglement with Trust(p)(p).

4.4 The Context Chain

State integrity in CP is maintained by the context chain—a cryptographic commitment to the complete succession history of each participant's context.

Definition 4.2 (Context Chain)**.** The anchor for participant p at height h is:

$$\alpha(h+1) = H\big(\mathrm{id}(p) \parallel (h{+}1) \parallel \alpha(h) \parallel \sigma(h{+}1)\big)$$

where H is a collision-resistant hash function, id(p) is the participant's unique identifier, and $\sigma(h{+}1)$ is the state at height $h{+}1$.

Each anchor depends on its predecessor, forming a self-attesting commitment to the participant's entire history. Given $\alpha(h)$ and a claimed history $(\sigma_0, \ldots, \sigma_h)$, the history can be verified in $O(h)$ hash operations. Any modification to any point in the history changes the current anchor. History is immutable by construction—not by policy, but by the cryptographic structure of the chain.

The context chain is the operational form of the KCC trajectory $\tau_p : \mathbb{R} \to T_pM$ from CC3's geometric model. Normal succession is participant-initiated: the participant's embedded witness set attests to each transition. No external validator is required. In case of witness unavailability, recovery returns to the network's genesis context—the participant's recovery authority of last resort (Section 4.9).

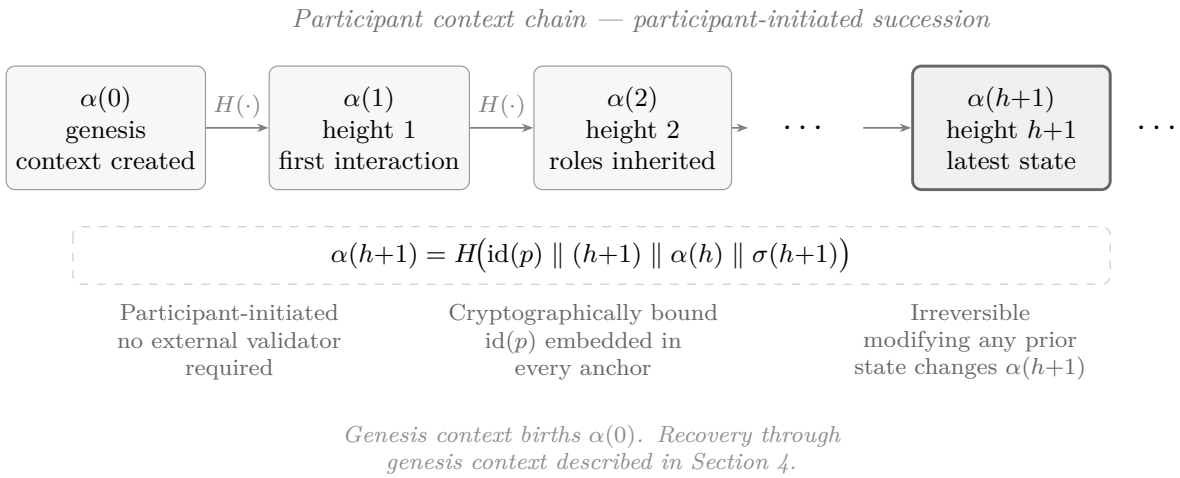

Figure 3: Context chain: participant-initiated succession, cryptographically bound, irreversible by construction. $\mathrm{id}(p)$ is embedded in every anchor, making the chain participant-indexed. Genesis context births $\alpha(0)$; recovery through genesis context described in Section 4.9.

4.5 Primary Context and Inherited Contexts

Every participant has exactly one primary context at any point in time. The primary context is the irreducible representation of WHO the participant is—their sovereign computational existence independent of any application or protocol they may participate in.

Participants may additionally inhabit inherited contexts, representing roles and shared computational spaces. Inherited contexts are explicit and stateful: a participant must actively acquire an inherited context, and this fact is recorded in their primary context's Trust sub-context. The participant can revoke this inheritance, and the fact of revocation is equally recorded.

Definition 4.3 (Inherited Context)**.** An inherited context C_i is a context that participant p inhabits in addition to their primary context $C(p)$. Inheritance is recorded linearly in $\mathrm{Trust}(p)(p)$ at a specific height with specific witnesses. It is not ambient—it is an explicit, participant-sovereign commitment. Two forms exist:

Role contexts—additional contexts representing organisational or functional roles, each carrying its own witness set, logic handlers, and interaction types. p's primary context remains free; personal operations proceed independently of role context activity.

Shared inherited contexts—contexts inhabited by multiple participants simultaneously (e.g. a market, a DEX, a governance space). Each participant brings their sovereign state to the shared context. The shared context provides shared computation rules and operational parameters, but it does not hold shared value state. There is no shared value pool.

Commutativity holds structurally: the condition that creates ordering dependence—shared value state—is absent.

The Cartesian conflation and the CP separation. In every system built on Cartesian foundations, shared computational space and shared state are inseparable. State is anonymous and globally indexed. When a shared computational space is created, state comes with it by default. The compute and the state occupy the same object. Participants writing to a shared space are writing to shared state. This conflation is what makes ordering necessary: shared compute implies shared state, shared state implies write conflicts, write conflicts imply ordering. The entire chain is forced—not by the requirements of the application, but by the structure of the substrate.

CP breaks this conflation structurally. State is participant-indexed. A shared inherited context C_i carries Logic(C_i) and Prefs(C_i)—computation rules and operational parameters. It does not carry value state. When multiple participants inhabit the same inherited context, they share the computation frame. They do not share state.

C_i is itself a participant with its own Context Superstate $C(C_i)$. It may hold its own Assets(C_i), Storage(C_i), Logic(C_i), Keys(C_i), Prefs(C_i), and Intel(C_i) as a sovereign participant in its own right. What is structurally non-expressible is for C_i to hold Assets(p) on behalf of another participant p. Value is participant-indexed by construction: the diagonal morphism does not exist, and there is no mechanism by which Assets(p) can be placed in $C(C_i)$.

The transition function for inherited contexts. For an interaction ι executing within inherited context C_i between participants p_1 and p_2:

$$\delta\big(C(p_1, C_i),\ C(p_2, C_i),\ \iota\big) = \big(C(p_1, C_i)',\ C(p_2, C_i)'\big)$$

where f_ι is drawn from Logic(C_i). The post-contexts reflect updated participant state. C_i itself is unchanged unless a governance action explicitly modifies it. Two participants operating within the same inherited context execute independently unless their interaction names both. The inherited context provides a shared computational frame. It does not create shared state.

A concrete example. Consider two participants each interacting with a shared market context M. Participant p_1 holds 100 units of asset A; participant p_2 holds 50 units of asset B. Both inherit M. In the case where each participant's interaction does not modify M's own state (e.g. a read-only price query), $C(M)$ is unchanged:

$$\delta\big(C(p_1, M),\ C(M),\ \iota_1\big) = \big(C(p_1, M)',\ C(M)\big)$$
$$\delta\big(C(p_2, M),\ C(M),\ \iota_2\big) = \big(C(p_2, M)',\ C(M)\big)$$

ι_1 and ι_2 touch only their own participant contexts. They execute in full parallel. No ordering. Not because independence was proved by static analysis—but because their state was never shared in the first place.

When interactions do modify M's state—as in a live AMM where the price curve updates after every trade—M is named as a full participant in the ICS:

$$\delta\big(C(p_1, M),\ C(p_2, M),\ C(M),\ \iota\big) = \big(C(p_1, M)',\ C(p_2, M)',\ C(M)'\big)$$

p_1's tokens move in $C(p_1, M)$. p_2's tokens move in $C(p_2, M)$. The price updates in $C(M)$—sovereign state indexed to M. A second trade naming $C(M)$ must operate on $C'(M)$. This ordering is local: resolved by M's witness set, finalised by IFP in 2Δ. No interaction not naming M is affected. This is not global ordering. It is participant-scoped ordering—the only kind that exists in CC Protocol.

What a shared inherited context can and cannot do. A shared inherited context C_i is itself a participant. It owns its own state as a sovereign participant in its own right. The owner of C_i can modify Logic(C_i)— update pricing rules, change interaction types, set operational parameters. This is a governance action, explicitly authorised by C_i's owner, recorded in C_i's context chain. It is categorically distinct from participant interactions within C_i.

What C_i cannot do is hold participants' assets, sequence participants' interactions against each other, or impose ordering on operations that touch only individual participant contexts. These are not restrictions imposed on C_i by the protocol. They are structural absences: there is no mechanism by which C_i could hold participant assets, because assets are participant-indexed and no operation transfers them to the context. There is nothing to sequence, because there is no shared state for ordering to protect.

The ordering requirement that Cartesian systems impose on shared spaces arises entirely from the conflation of shared compute and shared state. Separate them—as CP does structurally— and the ordering requirement dissolves. Not as an optimisation. As a categorical consequence of the correct state model.

This architecture makes interoperability native. The participant is the interoperability primitive. When value moves between contexts, it moves with and through the participant. No bridge protocol holds custody. No wrapped representation is required. The participant is the bridge.

The structural consequences of this separation—including the non-expressibility of flash-loan-style compositions—are developed fully in Section 5.

4.6 Context Succession

Context succession is the mechanism by which $C(p)$ evolves from height h to height $h{+}1$. It is the operational form of the KCC update step—the moment at which an interaction resolves and the participant's context advances.

Definition 4.4 (Context Succession)**.** A succession from $C(p)$ at height h to $C(p)$ at height $h{+}1$ is valid when:

1. A triggering event occurs (interaction commit, explicit request, or witness rotation).
2. The state transition $\sigma_h \to \sigma_{h+1}$ is validated by $\geq 2f{+}1$ witnesses.
3. The new anchor $\alpha(h+1) = H\big(\mathrm{id}(p) \parallel (h{+}1) \parallel \alpha(h) \parallel \sigma(h{+}1)\big)$ is computed.
4. Witnesses update their local attestations to the new anchor.

Succession is linear by construction. $C(p)$ at height $h{+}1$ commits to $C(p)$ at height h. There cannot be two distinct $C(p)$ states at height $h{+}1$, because each must commit to the same $C(p)$ at height h, and any two competing successions would require the witness set to attest to contradictory successors—which $2f{+}1$ honest witnesses will not do.

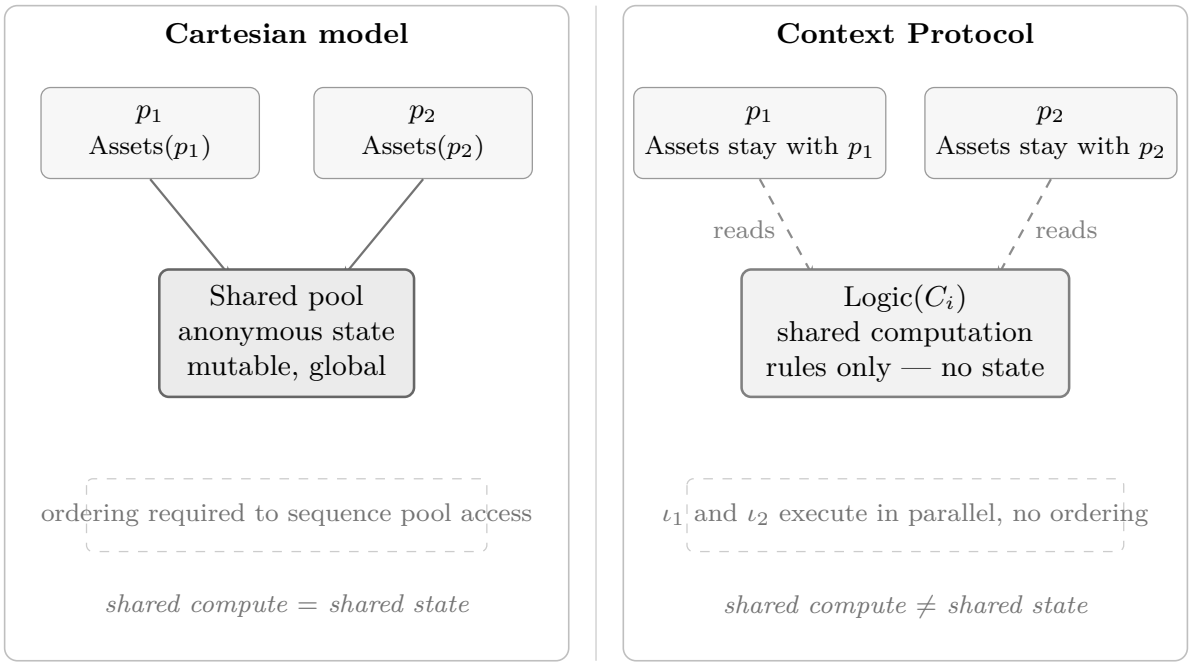

Figure 4: Shared compute vs shared state. Left: in Cartesian systems shared compute implies shared state—assets flow into an anonymous pool, requiring ordering. Right: in CP, Logic(C_i) is shared computation only; participant assets remain sovereign. Interactions ι_1 and ι_2 execute in parallel.

Context updates follow a lazy model: succession occurs on triggering events, not through continuous global synchronisation. This eliminates the overhead of maintaining globally fresh state for every participant at every moment—overhead that would reintroduce exactly the scalability constraints CP is designed to eliminate.

4.7 Context Genesis

Every participant context traces its lineage to a single origin. When a new participant joins, a genesis binding is established between the network and the new participant—not merely a registration record, but a genuine contextual binding that anchors the participant's existence within the network's frame of reference. The genesis binding establishes the participant's existence within the network, the participant's initial state at height 0, and a persistent anchor on the participant's context chain as cryptographic proof of network participation. The genesis anchor $\alpha(0)$ is the base case from which all subsequent anchors derive their chain integrity.

4.8 Interaction Context Space Formation

When participants interact, their contexts compose into the Interaction Context Space (ICS)—the total scope within which the interaction's transformation is defined and witnessed.

Definition 4.5 (Interaction Context Space)**.** The ICS for interaction ι is:

$$\mathrm{ICS}(\iota) = (P_\iota,\ C_\iota,\ \Phi)$$

where $P_\iota = S \cup T$ is the participant set (sources and targets), $C_\iota = \bigoplus_{p \in P_\iota} C(p)$ is the composite context formed by all named participants, and Φ is the witness requirement policy specifying the threshold and composition rule for the witness set $W(\iota)$.

Theorem 4.6 (Context Boundedness)**.** *For any interaction ι, the complete* $\mathrm{ICS}(\iota)$ *is fully determined before execution begins.*

Proof. The participant set P_ι is determined by the interaction specification. Witness sets $W(p)$ are embedded in $\mathrm{Trust}(p)(p)$ for each participant and committed in their context chains. No runtime discovery of participants or witnesses is required or permitted. □

The scope of agreement is constituted by the participants themselves—by who they named, what witnesses they carry, and what inherited context they inhabit. The ICS is a participant-constituted fact, available to all honest witnesses before a single message is sent.

4.9 Context Reset

When the witness quorum required for normal succession becomes unavailable for a participant, CP provides a recovery mechanism. From the moment a participant issues a reset, the protocol processes no further actions for that participant until the reset completes. The participant's context is frozen at its current valid height.

No partial state can exist at the moment of reset—linearity guarantees this. A transition either completed with sufficient attestation or it did not occur. The reset resolves to a new witness configuration anchored to that frozen height.

The specific mechanism by which the new configuration is established—including how the network coordinates recovery when context nodes are unreachable—is determined by the implementing network and specified in CC6. Context reset is expensive by design. Its existence is the safety net ensuring no participant context is permanently unrecoverable. Under normal operation it is never needed.

4.10 Protocol Invariants

A CC-compliant CP implementation must maintain the following invariants at all times.

Single context vector. Each participant has exactly one primary context at any height. No branching, no forking, no two valid states at the same height.

Continuous reachability. At any point in time, a participant's context is reachable—either through their current witness set (normal path) or through a recovery chain (reset path). The protocol must never leave a participant's context permanently unreachable.

Participant sovereignty. No context transition occurs without the participant's cryptographic authorisation, except under explicitly participant-initiated context reset. This is Consent (Axiom 7.2) made operational.

Witness-embedded state. $W(p)$ is part of $\text{Trust}(p)(p)$, committed in the context chain. Witnesses are not assigned at runtime from an external pool—they are part of who the participant is.

Immutable history. The context chain is append-only. No modification to any prior state is expressible in the protocol grammar.

4.11 Implications

The Context Protocol constitutes more than a state management layer. It constitutes a new category of digital existence.

Sovereign digital existence. The Context Superstate is not a profile or a credential. It is a multi-dimensional, behavioural, contextual, and accumulating representation of a participant—not a profile or a credential, but a participant-indexed computational identity that grows with every interaction. The participant is not identified by the system. They exist within it.

Differentiated compliance on the same network. Because compliance requirements follow the participant's inherited context rather than a global network policy, a single network can simultaneously serve participants under different regulatory regimes. GDPR governs European participants; DPDP governs Indian participants. Both operate on the same network, differentiated by context. No separate deployments, no jurisdictional forks.

Not one size fits all. A coffee transaction and a house purchase are not the same interaction. They should not carry the same latency, the same trust model, or the same witness requirement. In CP they do not—the witness requirement policy Φ is per-interaction, derived from the participants' own context configuration.

Portable records. A participant's healthcare records, credentials, financial history, and attestations are carried in their Context Superstate and travel with them across platforms, providers, and jurisdictions. No transfer protocol, no import/export, no issuer intermediary required.

Delegated authority at the interaction level. Delegation is not a blanket permission. It is a specific authority for a specific interaction type, committed in $\text{Trust}(p)$ at a specific height, revocable at any subsequent height. Granular, auditable, and participant-controlled.

Emergent participant-centric intelligence. As a participant accumulates interaction history, $\text{Intel}(p)$—the EPI substrate—becomes an increasingly precise representation of that participant's computational behaviour, preferences, and relationships. Classical AI builds models by searching for signal across vast undifferentiated datasets—the needle in the haystack. In CP, the needle is the participant's own context—already distilled, already contextualised, already indexed to WHO. The intelligence does not need to be found. It emerges naturally from the participant's own history. No global training run, no generalised model, no search problem. The Context Superstate is the model.

Observation

The structure of CP maps naturally to how participants develop over time. A participant's primary context is created at their first network interaction—their genesis event. From there, the participant grows their context chain independently through interactions, accumulating history, inheriting roles, and operating across multiple shared spaces simultaneously—each as a distinct inherited context, each sovereign. Native interoperability follows: the participant is the bridge between contexts, not a bridge protocol. And when a participant's witness set becomes unavailable, the genesis context provides the recovery path—always present, always reachable, supporting without governing.

This is how participation already works in the physical world. A person holds multiple roles simultaneously—professional, familial, civic, commercial. They carry their identity with them across contexts; they are not recreated by each context they enter. When they need support there is a primordial anchor: family, community, institution. People do not become new entities when they open a bank account or join a platform or enter a market. They bring themselves. CP provides the computational substrate that finally reflects this. The mathematical objects—the primary context, the inherited context, the context chain, the genesis recovery path—are not invented structures. They are observations. The formalism is pointing at something that was always true about how interaction works.

5 Interaction State Machine (ISM)

5.1 The Semantic Contract

The Interaction State Machine realises the interaction semantics of CC4. PIC defines the interaction 5-tuple $\iota = (S, T, W, f_\iota, \mathrm{Auth}_\iota)$ and establishes that f_ι is a linear transformation over the joint participant contexts of $S \cup T$. ISM specifies how that transformation is expressed and executed on a live network. It is the computation engine of the standard.

The central recognition ISM embodies: domain interactions are first-class protocol primitives, not application-layer conventions. A value transfer is not a function call to a contract that happens to move tokens. It is a typed operation with protocol-defined semantics, verifiable against protocol invariants, and executable without analysing arbitrary code paths. The grammar of value computation is constitutive—it determines what operations exist, not what operations are permitted.

5.2 The State Transition Function

Definition 5.1 (ISM State Transition)**.** For interaction ι over participant set $P_\iota = \{p_1, \ldots, p_n\}$:

$$\delta(C(p_1),\, C(p_2),\, \ldots,\, C(p_n),\, \iota) = (C(p_1)',\, C(p_2)',\, \ldots,\, C(p_n)')$$

Only the named participants' contexts enter the function. Only those contexts exit. The pre-contexts are consumed exactly once. The post-contexts are produced exactly once. No intermediate state exists.

The contrast with the classical model is structural, not notational. In the classical model: $\sigma' = \Upsilon(\sigma, T)$. One global state σ enters. One transaction T is applied. One new global state σ' exits. The scope of every transaction is the entire world state—because σ is the entire world state, every transaction reads and writes to the same global object. Any two transactions potentially conflict. The system cannot know before execution whether two transactions conflict. It must order them globally, totally, and execute them sequentially.

The scope problem is deeper than the ordering problem. Because σ is global and anonymous, the system has no structural way to determine the natural boundary of any transaction. A payment between Alice and Bob has the same formal scope as a governance action affecting the entire network—both are functions from σ to σ'. The protocol cannot distinguish them structurally. It can only inspect their code and infer their effects, which is why static analysis and access list declarations have become standard practice in high-performance classical systems. These are workarounds for a scope problem that the substrate introduces by construction.

In ISM, the scope of every interaction is determined before execution begins. The named participants define the scope. Only their contexts enter and exit the transition function. Everything else in the network is provably untouched—not by inspection or analysis, but by the structure of the function. The scope is constitutive, not restrictive.

This is the observation at the heart of ISM: the natural boundary of a value interaction is the set of participants it names. That boundary was always there. The classical model

ignored it because it had no substrate for participant-indexed state. ISM formalises what was always true.

The cost of any interaction is $O(k \times t)$ where $k = |P_\iota|$ is the number of named participants and t is the interaction complexity. Network size N does not appear in the cost function.

The complexity consequence is not a performance claim. It is a structural separation between two categorically different computation models.

In a system where all validators must process all operations, the cost per participant is $O(n \times c)$ where n is network size and c is the average number of operations per participant. Total system cost is $O(n^2 \times c)$. This holds regardless of parallelisation technique, sharding strategy, or hardware acceleration. It is a consequence of the requirement that all validators maintain consistent knowledge of all state.

In ISM, each participant context requires processing only for operations that directly affect it. A participant with k interaction partners is affected by $O(k \times c)$ operations. Because contexts are independent, the validators for each context process only that context's operations. Total system cost is $O(n \times k \times c)$. When k is small relative to n—as it always is in real systems where participants interact with a bounded subset of others—this is a different complexity class, not a constant-factor improvement.

More precisely: in the Cartesian model, throughput is bounded by $O(r \log n)$ regardless of available computational resources, where r is the operation rate per participant. In ISM, throughput scales as $O(n \times r)$—linear with the number of participants— because operations over disjoint contexts execute in full parallel with no ordering dependency between them.

This is not an optimisation of the global ordering model. It is what computation looks like when the participant is a primitive dimension, not a derived construct assembled from anonymous records.

When participant p inhabits inherited context C_i:

$$\delta\big(C(p_1, C_i),\ C(p_2, C_i),\ \iota\big) = \big(C(p_1, C_i)',\ C(p_2, C_i)'\big)$$

Logic(C_i) governs the permissible transformations. Assets remain in $C(p)$. The shared context computes over participant state. It does not hold it.

5.3 The Two-Bucket Model

Every interaction in ISM operates over two categorically distinct inputs.

The **value bucket** contains linear state—the participant contexts and the value objects within them. Value bucket contents are governed by the Linearity axiom: consumed exactly once, cannot be duplicated, cannot be discarded without explicit retirement. Conservation holds across the value bucket for every interaction without exception.

The **information bucket** contains Cartesian data—developer-supplied parameters, oracle inputs, configuration values, and any other data that accompanies the interaction but does not itself carry linear value. Information bucket contents are freely readable and do not carry ownership semantics.

The separation is structural. Any pattern that requires treating value as information—borrowing it, using it to manipulate state, and returning it within a single atomic frame

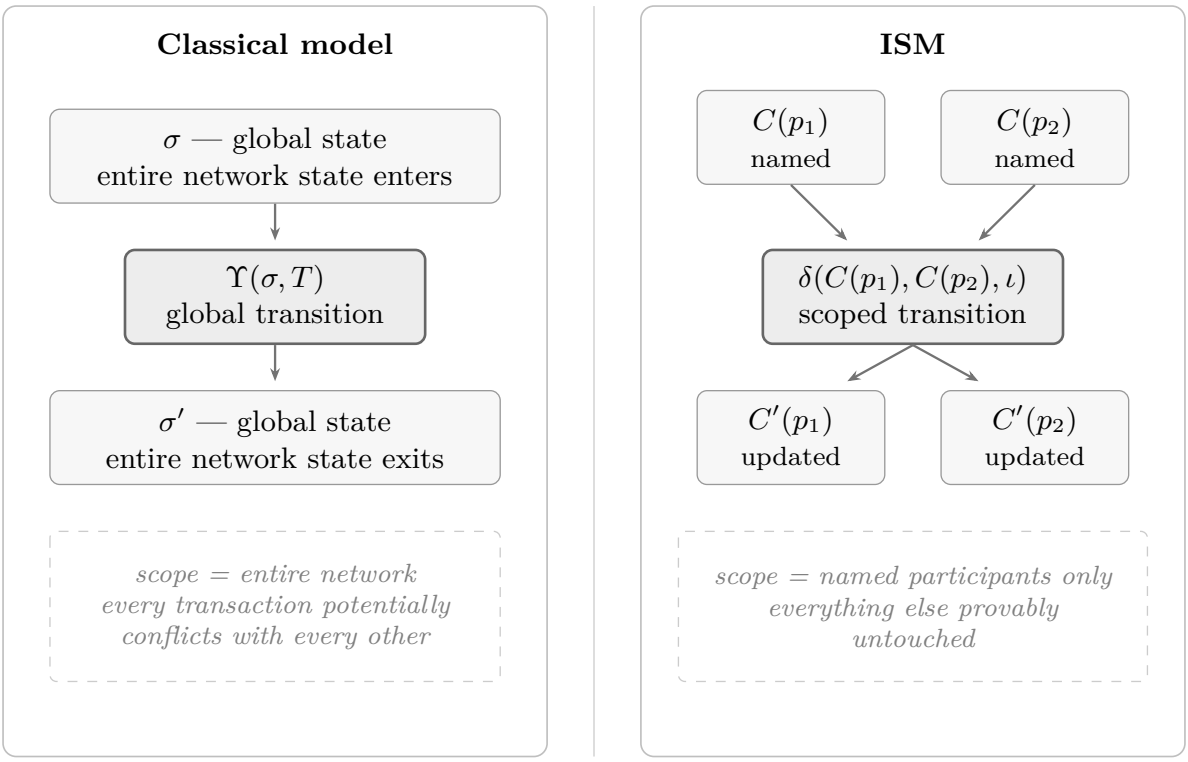

Figure 5: ISM vs classical model. Left: $\Upsilon(\sigma, T)$ takes the entire global state in and produces the entire global state out. Right: δ takes only named participant contexts in and produces only those contexts out. Everything else is provably untouched. Scope is constitutive.

spanning unrelated participants—requires a diagonal morphism on value objects. The diagonal morphism does not exist in V_{PIC}. The operation is absent from the algebra, not caught at runtime.

The flash loan is one well-known instance of this pattern. It requires borrowing value, using that value as information to manipulate state in unrelated contexts, and returning the value—all within a single atomic frame. This requires treating value as information (crossing from the value bucket to the information bucket), composing across unrelated participant sets (violating locality), and expressing a diagonal morphism on value objects (which does not exist in V_{PIC}). Every step of the composition is absent from the ISM grammar. It is undefined not because it is detected and rejected, but because the grammar that would express it does not exist. This is the difference between a prohibition and an absence. Prohibitions can be circumvented. Absences cannot.

5.4 Typed Interaction Domains

ISM defines typed interaction domains—protocol-recognised categories of transformation with well-defined semantics, deterministic state transitions, and bounded execution paths. Elevating an interaction type to a domain primitive has three consequences: the implementation is shared and protocol-verified, reducing audit burden; the attack surface contracts; and proof generation becomes tractable, because execution paths are bounded and deterministic.

A CC-compliant ISM implementation must support at minimum the following domain structure:

Participant domain. Operations governing the lifecycle of participant contexts— genesis, succession, witness rotation, context reset, inherited context management. These are the WHO dimension managed at protocol level.

Asset domain. Operations governing native value objects—creation, transfer, issuance, locking, delegation, and retirement. Assets in a CC-compliant system are protocol-level objects, not application-layer conventions. The protocol knows what a transfer means and enforces conservation invariants directly. Logic never mutates asset state directly; it requests operations through the asset execution layer, which enforces linearity at the type level.

Logic domain. Operations governing participant-owned computation—handler deployment, versioning, and invocation. Logic executes within participant contexts, not as global state mutations.

Extension mechanism. New interaction domains may be introduced by implementing networks without altering fundamental execution semantics or violating the two-bucket model. Each new domain type is simultaneously a new protocol capability and a new bounded execution path.

5.5 Structural Properties

Four structural properties hold for every ISM execution without exception, derived directly from PIC's axioms.

Atomicity. No intermediate state exists between pre- and post-interaction contexts. f_ι either executes completely or does not execute at all. Because value is linear, there is nothing to partially consume.

Linearity. Each $C(p) \in C_\iota$ is consumed exactly once. No context is duplicated, forked, or partially consumed. This is a property of the execution environment's type structure, not a runtime check. The operation of copying a context does not exist in the ISM grammar.

Locality. f_ι operates exclusively over $\{C(p) \mid p \in P_\iota\}$ and their witness set W_ι. No other context is readable or writable during execution. Locality in ISM is logical and witness-bounded—defined by WHO is in the transformation, not WHERE they are geographically.

Determinism. Given the same ICS and the same interaction specification, f_ι produces identical outputs. All information required to determine the outcome is present in the ICS before execution begins. No external oracle or global state query is required mid-execution. The execution environment contains no non-deterministic primitives.

5.6 Re-entrancy: The Structural Argument

Re-entrancy is not prevented in ISM. It is structurally non-expressible. Two independent layers make this so.

At the *type layer*: $C(p)$ is consumed at the moment execution begins. The pre-context does not persist through execution. A callback cannot re-enter a function and read pre-

write state because the pre-write state has been consumed. There is no state to re-enter.

At the *execution layer*: a context currently being consumed cannot be re-referenced by any other operation within the same execution scope. The locality property makes this structural.

Either layer alone eliminates re-entrancy. Together they make it non-expressible at two independent levels—absent from the grammar rather than caught at runtime. This is the distinction between a prohibition and an absence: a re-entrancy check detects and rejects. ISM has no mechanism to reject re-entrancy because there is no mechanism by which re-entrancy can be expressed.

5.7 Network Objects: The First Linear Primitive at Network Level

Between PIC's formal semantics and ISM's operational execution, a new primitive is required. Classical networks are built on Cartesian primitives: objects are copyable, state is replicable. The correct primitive for value computation is different in kind.

Definition 5.2 (Network Object)**.** A Network Object is a computational element that is:

- **Stateful**—maintains persistent participant-indexed state;
- **Linear**—cannot be copied; linearity is a property of the object itself, not a constraint imposed upon it;
- **Participant-bound**—entangled with a specific WHO;
- **Witness-embedded**—carries its agreement proof within the object itself;
- **Deterministic**—transitions produce identical outputs given identical inputs.

A Context Superstate $C(p)$ is a Network Object. Network Objects are the state primitives that ISM's transition function δ operates on.

The distinction between a record in a replicated database and a Network Object is ontological. A database record answers: "what is Sally's balance?" A Network Object answers: "what is Sally's *agreed* balance?"—value entangled with a specific participant within a specific context, carrying embedded witness proofs, deterministically knowable without external consensus. These are not two answers to the same question. They are answers to two different questions. Network Objects are the first primitive for which linearity is a property of the primitive itself, not a constraint imposed on top of a Cartesian substrate. Classical distributed systems were built to answer the first. ISM is built to answer the second.

5.8 ISM vs Classical Model: A Structural Comparison

Table 2: ISM vs classical transition model

	Classical model	**ISM**
Input state	σ — the entire world state	$C(p_1), \ldots, C(p_n)$ — named participants only
Output state	σ' — the entire world state	$C(p_1)', \ldots, C(p_n)'$ — named participants only
Parallelism	Sequential — same σ for every T	Structural — disjoint P_ι means disjoint state
Linearity	Not enforced — σ is freely copyable	Structural — $C(p)$ consumed on transition
Conflict detection	Runtime — requires global ordering	Structural — scope determines conflicts before execution
Finality scope	Global — entire network must agree	Local — $P_\iota \cup W_\iota$ is sufficient
ZK proof cost	Generalised ZKVM — unbounded	Typed operation — bounded, predictable

The distinction is not merely structural. It is epistemological—it determines what the system can know about itself before any message is sent.

5.9 Compliance Requirements for ISM

A CC-compliant ISM implementation must satisfy the following normative requirements. The state transition function must be of the form $\delta(C(p_1), \ldots, C(p_n), \iota) = (C'(p_1), \ldots, C'(p_n))$—scoped to named participant contexts only. No global state may enter or exit the function. The four structural properties—atomicity, linearity, locality, and determinism—must hold for every execution. The domain model must include at minimum the participant, asset, and logic domains, with an extension mechanism. Re-entrancy must be non-expressible—eliminated at the type level, not merely detected at runtime. No inherited context may hold value state on behalf of participants. The specific programming language, virtual machine, and asset standard implementations are implementation choices available to CC-compliant networks, documented for MOI in CC6.

5.10 Implications

The Interaction State Machine constitutes a new execution model for value computation.

On-demand state management. State exists only when an interaction creates it. No pre-deployment of contracts, no persistent global state objects sitting idle. A participant's context is updated only when they interact. The network does not maintain globally fresh state for every participant at every moment.

Typed operations as building blocks. ISM interaction operations are composable typed primitives. Participants and developers compose them to build any interaction pattern: payment, escrow, governance, identity verification, asset issuance, AI agent coordination. Not always a smart contract. The operation is the primitive, not the contract. Contracts on Cartesian systems are general-purpose programs that happen to move value. ISM operations are value-native typed primitives that happen to be composable.

Change management without migration. Upgrading logic on a Cartesian system requires migrating state, managing versions, and coordinating all users simultaneously. In ISM, Logic(p) is participant-owned. A participant updates their own logic handler without touching anyone else's state. Change is local and non-disruptive. The same architecture makes integration with existing systems natural: Web2 APIs, enterprise databases, and legacy infrastructure feed into the information bucket as inputs to participant logic. The participant is the integration point. No bridge protocol, no adapter layer, no state migration required.

ZK tractability. Typed interaction operations with known input and output shapes make ZK proof generation cheap and predictable. No generalised ZKVM required. Proof verification cost is a function of the operation type, not of arbitrary code complexity. Because the participant anchor is embedded in every Network Object, proofs are context-specific: you prove about a specific participant's context transition, not about global state.

Industry-specific interaction domains. New ISM interaction domains can be defined for specific domains: healthcare interactions with embedded regulatory compliance, AI agent coordination with participant-anchored accountability, legal instruments with delegated authority, supply chain events with provenance tracking. Each with protocol-level semantics and the security guarantees that come from the participant anchor. Not custom contracts sitting on top of a generalised VM. Native protocol operations.

5.11 The Elimination of Global Ordering

Every distributed system built on Cartesian foundations has one ordering scope: global. State is anonymous and globally indexed. Identity—the account, the address, the owner field—is metadata attached to anonymous records. It determines who may sign a transaction. It does not change what the state space is. The state space remains global and anonymous regardless of how ownership is labelled. Global ordering is therefore structural: the only available authority when two operations contest the same anonymous state is the full network. This holds regardless of how many objects are owned, how many transactions execute in parallel, or how sophisticated the parallelisation strategy. The moment any shared state is touched, the global ordering authority is invoked. That moment arrives precisely when multi-party applications form.

Object-ownership models reduce the frequency with which global ordering is invoked. Owned objects—objects with a single designated owner—can execute in parallel because they touch disjoint state. This is a real improvement. But the substrate has not changed. The state space is still global and anonymous. Shared objects—price curves, lending pools, governance registries—still require global consensus: processed by the same validator set, ordered in the same global DAG, and checkpointed together. Execution of non-conflicting shared objects can be parallelised after ordering, but the ordering authority is always

the full network. The global ordering infrastructure—same validators, same DAG, same checkpoint—is always present for any shared object, regardless of whether it conflicts with another. The global ordering authority is deferred, not dissolved. And it is deferred only until the moment an application requires shared state—which is to say, until the application does anything interesting.

The CC Protocol does not reduce the scope of global ordering. It eliminates the global ordering scope entirely.

There is no anonymous global state in a CC-compliant network. State is participant-indexed by construction. What appears to be a shared space—a market, a DEX, a governance forum—is a participant M with its own Context Superstate $C(M)$, its own witness set embedded in $\mathrm{Trust}(p)(M)$, and its own context chain recording every transition it has undergone. M's price lives in $C(M)$—sovereign state indexed to M. There is no anonymous price object. There is no shared pool. There is only a participant whose state records the current price, and interactions that name that participant.

When p_1 and p_2 trade within market M, they have each statefully inherited $C(M)$—this inheritance is recorded in their own context chains at a specific height with specific witnesses. The ICS for their interaction is:

$$\mathrm{ICS}(\iota) = C(p_1, M) \oplus C(p_2, M) \oplus C(M)$$

Every element is participant-indexed. p_1's tokens are in $C(p_1, M)$. p_2's tokens are in $C(p_2, M)$. The price is in $C(M)$. The state transition:

$$\delta\big(C(p_1, M),\ C(p_2, M),\ C(M),\ \iota\big) = \big(C'(p_1, M),\ C'(p_2, M),\ C'(M)\big)$$

produces updated sovereign state for each named participant. No anonymous object is written. No global state is modified. The global network is not consulted.

When a second trade ι_2 between p_3 and p_4 names the same $C(M)$, an ordering requirement exists—ι_2 must operate on $C'(M)$, the post-state of ι_1. This is not global ordering. This is IFP running within the ICS that names M, resolved by M's witness set, finalised in 2Δ. The ordering authority is M's witnesses—participant-constituted, interaction-scoped, already determined before the interaction begins because the witness set is embedded in $\mathrm{Trust}(p)(M)$. No interaction not naming M is affected. No network-wide agreement is sought. The ordering is local by construction, because M is a participant, and participants have local witnesses, and local witnesses produce local finality.

A participant p may simultaneously inhabit multiple inherited contexts—a lending protocol $C(L)$, a market $C(M)$, a governance space $C(G)$—and engage in direct value transfers outside any inherited context. Each proceeds in parallel at the network level. Interactions within $C(L)$ are ordered only among interactions that name $C(L)$, resolved by L's witnesses. Interactions within $C(M)$ are ordered only among interactions that name $C(M)$, resolved by M's witnesses. Direct value transfers require no ordering at all. None of these affect each other. They never will.

An application that creates unnecessary shared participant state introduces unnecessary ordering within its own ICS. That is an application design choice, not a protocol constraint. The protocol introduces no ordering dependency beyond what the named participant set requires. In Cartesian systems, the protocol always introduces global ordering regardless of what the application requires. In CC Protocol, the protocol introduces no

ordering at all. Ordering exists only where participant state is shared, is always scoped to that participant's witness set, and is always locally final.

The difference is not architectural. It is ontological. In Cartesian systems, identity is metadata. The state space is global. Global ordering is inescapable. In CC Protocol, the participant is the state. There is no global state space. There are no shared spaces—only participants and the interactions that name them. Global ordering is not reduced. It is absent.

Observation

The natural boundary of any value interaction is the set of participants it names. Every value exchange that has ever occurred had a natural scope—the people involved in it. Classical distributed systems erased that scope by building on anonymous global state. ISM restores it. The formalism is not introducing a constraint. It is recognising a structure that was always there.

6 Interaction Finality Protocol (IFP)

6.1 The Semantic Contract

The Interaction Finality Protocol realises the witness semantics of CC4. PIC establishes that value exists if and only if all constituents of an interaction participate—the Value–Agreement Equivalence (Theorem 5.11 in CC4). It establishes that agreement is constituted by participant witnessing, not discovered through external validation. It establishes that finality is intrinsic—constituted at the moment of witness attestation, not accumulated through confirmation depth. IFP is the protocol by which the constituent set $C_\iota = P_\iota \cup W_\iota$ reaches the attestation threshold required for that transformation to be final.

What PIC does not specify is how attestation is reached in practice. CP has constituted the ICS—the composite context formed by the named participants, their witnesses, and the computation rules of any inherited context they inhabit. ISM has computed the transformation—the post-contexts that would result from the interaction. IFP is the protocol by which the constituent set reaches the threshold required for that transformation to be final.

IFP solves a precisely bounded problem. It is not global consensus. It is not ordering. It is agreement within a specific, pre-determined ICS on which of potentially several competing proposed context transitions commits. The scope is fixed before execution begins. The constituent set is known. The question is singular: which proposed transition wins?

This precision is what makes the Triple Achievement possible.

6.2 The Triple Achievement

Traditional consensus protocols face a structural tradeoff among three properties: leaderless (no single validator coordinates; all participants symmetric), deterministic (given the same inputs, all honest validators reach the same decision), and fast (finality achieved in bounded time after GST). These three properties appear mutually exclusive within the global consensus model. Leader-based protocols sacrifice leaderlessness for speed. Probabilistic protocols sacrifice determinism for leaderlessness. View-change protocols attempt recovery from leader failure but add latency. The tradeoff is real — within that substrate.

Theorem 6.1 (Triple Achievement)**.** *IFP is simultaneously leaderless, deterministic, and achieves finality in bounded time after GST.*

IFP achieves all three simultaneously not by engineering a more efficient consensus protocol, but by changing the problem domain. Leaderlessness follows because there is no global sequence to impose—only a local conflict set to resolve. Determinism follows because the conflict resolution rule is a deterministic function of inputs known to all constituents before execution begins. Bounded finality follows from monotonic elimination within a finite totally ordered set. The three properties are not in tension. They are consequences of the same structural decision: scope the agreement problem to the ICS.

6.3 ICS Constitution

Before IFP can execute, the scope of agreement must be precisely and completely determined.

Definition 6.2 (ICS — IFP View)**.** The ICS for interaction ι is $\mathrm{ICS}(\iota) = (P_\iota, C_\iota, \Phi)$ where $P_\iota = S \cup T$ is the participant set, $C_\iota = \bigoplus_{p \in P_\iota} C(p)$ is the composite context at the current height of each named participant, and Φ is the witness requirement policy—the threshold and composition rule governing the witness set $W(\iota)$.

Theorem 6.3 (Context Boundedness)**.** *For any interaction ι, the complete* $\mathrm{ICS}(\iota)$ *is fully determined before execution begins. The participant set is determined by the interaction specification. Witness sets are embedded in* $\mathrm{Trust}(p)$ *for each participant and committed in their context chains. No runtime discovery is required.*

Proof. The participant set P_ι is determined by the interaction specification. Witness sets $W(p)$ are embedded in $\mathrm{Trust}(p)$ for each participant and committed in their context chains. No runtime discovery of participants or witnesses is required or permitted. □

This is not a performance property. It is structural. The scope of agreement is constituted by the participants themselves—by who they named, what witnesses they carry, what inherited context they inhabit. The ICS is a participant-constituted fact, available to all honest constituents before a single message is sent.

A valid ICS must satisfy four conditions before the protocol proceeds:

1. All named participants $C(p)$ at their current heights are available and context-chain-verifiable.
2. The witness set $W(\iota)$ satisfies $|W(\iota)| = 3f+1$ for some $f \geq 1$.
3. All source participants have provided cryptographic authorisation Auth_ι.
4. The transformation f_ι is well-formed and deterministic.

If any condition fails, the ICS is invalid and execution does not proceed. This is not a runtime rejection. It is a pre-execution structural check.

6.4 The Four Phases

IFP operates in four sequential phases. The first two are convergent and tolerate asynchrony. The third and fourth execute under partial synchrony and achieve finality in bounded time after GST.

Preparation and Formation are the asynchrony-tolerant availability layer. Confirmation is the leaderless deterministic core that persists across views. Commit is finality—not a phase with duration, but a threshold event.

6.4.1 Phase 1 — Preparation

When a constituent $v \in W(\iota)$ receives a valid ICS X, it verifies three conditions: are the pre-contexts valid at the claimed heights with matching context-chain anchors? Does the

Table 3: Three-layer temporal structure of IFP

Layer	**Temporal character**	**Tolerance**
Preparation + Formation	Convergent, accumulates across rounds	Tolerates asynchrony—waits for $2f+1$
Confirmation	Local, monotonic, view-persistent	Tolerates view timeouts—state never resets
Commit	Single irreversible moment	Deterministic once both conditions met

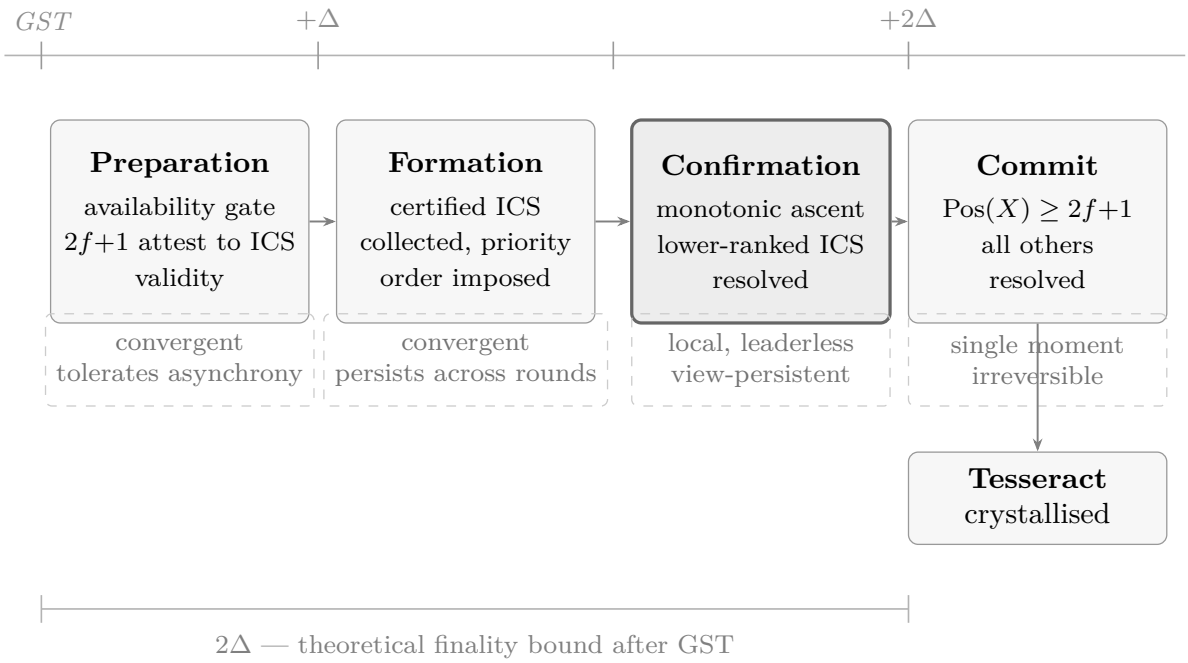

Figure 6: IFP four phases: Preparation and Formation are convergent and tolerate asynchrony. Confirmation is local, leaderless, and view-persistent. Commit is a single irreversible moment. 2Δ is the theoretical finality bound after GST.

transformation satisfy conservation across the value bucket? Have all source participants provided cryptographic authorisation?

If all conditions hold, v broadcasts a signed attestation. An ICS becomes *certified* when $2f+1$ constituents have attested to its validity and availability. Preparation is the availability gate: only certified interactions enter the confirmation protocol. An interaction that cannot achieve certification is not a conflict to be resolved; it is an invalid proposal that is absent from the grammar.

If a single honest constituent holds a certificate, then at least $f+1$ honest constituents have seen and verified the ICS. Under partial synchrony, every honest constituent eventually obtains the same certificate. The certified conflict set is therefore identical at all honest constituents after GST.

Preparation is convergent: a constituent accumulates attestations over time and tolerates asynchrony precisely because it waits until $2f+1$ attestations are collected regardless of

how long that takes. Under partial synchrony, all honest constituents eventually certify the same set of valid ICS proposals.

6.4.2 Phase 2 — Formation

Definition 6.4 (Conflict)**.** Interactions ι_1 and ι_2 conflict if:

$$\exists\, p \in (P_{\iota_1} \cap P_{\iota_2}) \text{ such that both propose modifications to } C(p).$$

Constituents collect all certified ICS proposals competing for the same participant contexts in the same execution slot. The resulting conflict set $S = \{X_1, X_2, \ldots, X_k\}$ is given a deterministic total order:

$$\iota_1 \prec \iota_2 \iff H(\iota_1 \,\|\, R) < H(\iota_2 \,\|\, R)$$

where R is slot randomness derived from the previous context chain height. This ordering is deterministic, unpredictable in advance, and local to the conflict set. This is conflict resolution, not ordering. Formation is convergent: the certified conflict set S becomes identical at all honest constituents eventually under partial synchrony. Confirmation begins as soon as a constituent has a non-empty conflict set and proceeds as additional attestations arrive.

Lemma 6.5 (Formation Convergence)**.** *Under partial synchrony, all honest constituents eventually reach identical conflict sets S with identical priority orderings.*

Proof. Under partial synchrony, after GST all messages are delivered within Δ time. All honest constituents certify the same valid ICS proposals (by the $2f+1$ threshold and Byzantine fault bound). The priority ordering is a deterministic function of certified proposals and public slot randomness R. Therefore all honest constituents compute the same S and the same ordering. □

6.4.3 Phase 3 — Confirmation

The monotonic confirmation protocol is the core of IFP. It is leaderless, deterministic, and operates over the finite conflict set S with deterministic total order $X_1 \prec X_2 \prec \cdots \prec X_k$.

Each constituent $v \in W(\iota)$ maintains local state: cur_v (current vote, initialised to $\perp$), $\mathrm{Pos}(X)$ (support count for X), $\mathrm{Neg}(Y)$ (rejection count for Y), and $\mathrm{status}(X) \in \{\text{live, resolved}\}$ for each $X \in S$.

Lemma 6.6 (Monotonic Ascent)**.** *For any honest constituent v, cur_v never decreases in the priority ordering. Once v votes for X, it never votes for any $Y \prec X$.*

Proof. v updates cur_v only when $\mathrm{Best}_v \succ \mathrm{cur}_v$. The condition is strict: the new vote must be strictly higher priority than the current vote. The algorithm contains no path by which cur_v can decrease. □

Lemma 6.7 (Rejection Monotonicity)**.** *Once constituent v broadcasts $\textsc{Reject}(Y)_v$, it never subsequently votes for Y.*

Algorithm 1 Monotonic Confirmation Protocol

```
upon receiving any VOTE or REJECT message do
Update Pos(X) and Neg(Y) from received messages
for each X ∈ S do
    if Neg(X) ≥ f+1 then
        status(X) ← resolved
    end if
end for
Best_v ← highest-priority live interaction in S
if Best_v ≻ cur_v then
    cur_v ← Best_v
    Broadcast VOTE(Best_v)_v
    for each Y ∈ S where Y ≺ Best_v do
        Broadcast REJECT(Y)_v
    end for
end if
if Pos(X) ≥ 2f+1 and ∀Y ≠ X : Neg(Y) ≥ f+1 then
    Commit X → produce Tesseract T(ι)
end if
```

Proof. v broadcasts $\textsc{Reject}(Y)_v$ only when moving its vote to some $Z \succ Y$. By Lemma 6.6, cur_v can only increase. Since $Y \prec Z \preceq \text{cur}_v$ for all future states, Y can never become Best_v. □

Lemma 6.8 (Resolution is Permanent)**.** *Once* $\text{status}(X) = \text{resolved}$*, it remains resolved. No honest constituent ever votes for a resolved interaction.*

Proof. $\text{status}(X) = \text{resolved}$ is set when $\text{Neg}(X) \geq f{+}1$. Since at most f constituents are Byzantine, at least one honest constituent has broadcast $\textsc{Reject}(X)$. By Lemma 6.6, that constituent's vote has moved to a higher-priority interaction and will not return to X. The resolved status is monotone: the condition $\text{Neg}(X) \geq f{+}1$ cannot become false once true. □

Confirmation is view-persistent: constituent state—cur_v, Pos, Neg, status—is never reset between rounds. If a view times out and a new view begins, the confirmation state carries over. Votes and rejections accumulated in prior views persist. A constituent never loses ground it has already gained in the climb. This eliminates the view-change overhead that characterises leader-based BFT protocols.

6.4.4 Phase 4 — Commit

Definition 6.9 (Commit Rule)**.** Interaction X commits when:

$$\text{Pos}(X) \geq 2f{+}1 \quad \text{and} \quad \forall Y \in S \setminus \{X\} : \text{Neg}(Y) \geq f{+}1$$

When X commits, the Tesseract $\mathcal{T}(\iota)$ is produced atomically across all participant context chains named in P_ι. The pre-contexts are consumed. The post-contexts are written. The commit proof Π — a set of $2f{+}1$ signed attestations — is embedded in the Tesseract. Both

conditions together guarantee that no other interaction can now achieve $2f+1$ support—the second condition ensures all alternatives are permanently resolved. Commit is a single irreversible moment: once both conditions are met, finality is the completion of the commit rule, not a property that accumulates afterward. Finality is immediate and permanent.

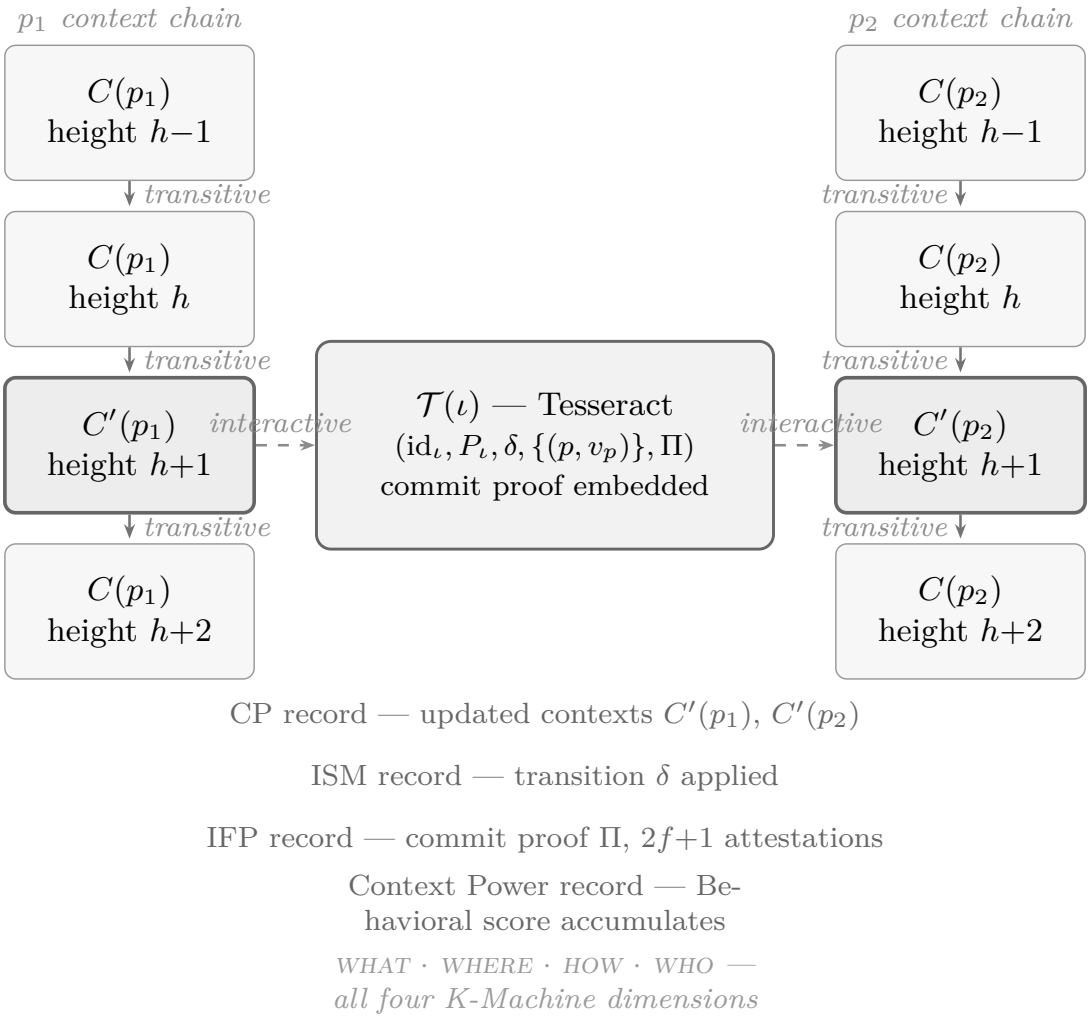

Figure 7: Tesseract $\mathcal{T}(\iota)$: the interactive edge connecting participant context chains at height $h+1$. Simultaneously a CP, ISM, IFP, and Context Power record. The commit proof Π is embedded within the Tesseract.

6.5 The Tesseract

When IFP commits, the result is crystallised as a Tesseract.

Definition 6.10 (Tesseract)**.** The Tesseract produced by committing interaction ι is:

$$\mathcal{T}(\iota)(\iota) = \big(\mathrm{id}_\iota,\ P_\iota,\ \delta,\ \{(p, v_p) : p \in P_\iota\},\ \Pi\big)$$

where $\mathrm{id}_\iota = H(\iota)$ is the unique interaction identifier, δ encodes the state transitions applied to each participant's context, (p, v_p) is the updated context record for each participant

p at height $h{+}1$, and Π is the commit proof—the $2f{+}1$ constituent attestations that constitute finality.

The Tesseract is named for the four dimensions of the K-Machine: WHAT, WHERE, HOW, and WHO—crystallised into a single irreducible record. It is the unit of settled value. Every property established by CP, ISM, and IFP is crystallised in it: the participant-indexing of CP, the linear state transition of ISM, and the Byzantine-tolerant agreement of IFP.

Bilateral. $\mathcal{T}(\iota)(\iota)$ records the updated contexts of all participants in P_ι simultaneously. No participant's post-context exists without the others. The transition is joint or it does not occur.

Final. The pre-contexts at their heights no longer exist. There is no shared reference frame within which the interaction could be revised. The Tesseract is the interaction's permanent record—not confirmation that it happened, but the object that constitutes its having happened.

The interactive edge. $\mathcal{T}(\iota)(\iota)$ is simultaneously present in every participant's context chain. The Tesseract is the node at which both edge types meet: transitive edges link successive context states vertically within each participant's own chain; interactive edges link Tesseracts horizontally across different participants' chains. It is the cross-reference that links Alice's context history to Bob's at the moment their interaction resolved. It is globally verifiable—any node can verify Π without having participated in the agreement.

The Tesseract is a CC Protocol object. How Tesseracts are stored, linked, and traversed within specific data structures belongs to implementing networks and is specified in CC6.

6.6 Safety and Liveness

All statements are scoped to a single interaction ι with constituent set $W(\iota)$ where $|W(\iota)| = 3f{+}1$ and at most f constituents are Byzantine.

Theorem 6.11 (Safety)**.** *No two honest constituents commit different interactions from the same conflict set S.*

Proof. Suppose v commits X and v' commits $Y \neq X$. For v to commit X: $\mathrm{Pos}(X) \geq 2f{+}1$, meaning at least $f{+}1$ honest constituents voted for X. For v' to commit Y: $\mathrm{Neg}(X) \geq f{+}1$, meaning at least one honest constituent rejected X. But by Lemma 6.6, an honest constituent that voted for X never rejects X. Contradiction. □

Theorem 6.12 (Validity)**.** *If honest constituent v commits interaction X, then $X \in S$—only certified interactions may be committed.*

Proof. The confirmation protocol is defined over S. Votes and rejections are generated only for elements of S. For any $X \notin S$, $\mathrm{Pos}(X) = 0$ globally. The commit rule requires $\mathrm{Pos}(X) \geq 2f{+}1$—impossible. □

Theorem 6.13 (Liveness)**.** *After GST, IFP commits some interaction from every non-empty conflict set S within 2Δ time.*

Proof. After GST all messages are delivered within Δ. By Lemma 6.5, all honest constituents reach identical S and identical priority ordering within Δ. Let X^* be the highest-priority interaction in S. All honest constituents set $\text{cur}_v \leftarrow X^*$ and broadcast $\textsc{Vote}(X^*)$ and $\textsc{Reject}(Y)$ for all $Y \prec X^*$. Within another Δ, all honest constituents receive $2f+1$ votes for X^* and $f+1$ rejections for all $Y \neq X^*$. The commit rule is satisfied. Total time: 2Δ. □

The 2Δ bound is the theoretical consequence of the leaderless monotonic confirmation model under partial synchrony after GST. Implementing networks that use coordination-assisted execution satisfy the same safety and liveness properties with latency characteristics that depend on the coordination mechanism. The 2Δ bound is the target the model converges toward as coordination assistance is progressively removed.

6.7 Conflict Resolution Is Not Ordering

This distinction is categorical and must be stated precisely, because it is the point at which IFP departs from every prior consensus model.

Ordering asks: in what sequence do we process all transactions on shared state? It presupposes a global shared object and asks who gets to write to it next. The answer requires a total order over all operations—which requires either a trusted coordinator or a globally elected one.

IFP asks: of the certified interactions competing for the same participant context in this execution slot, which one commits? This is a binary elimination question over a finite bounded set. The answer requires deterministic priority within the conflict set—which requires no coordinator at all.

Non-conflicting interactions—interactions whose participant sets are disjoint—are not ordered relative to each other. They are not sequenced. They are not even aware of each other. They execute in full parallel because they have no causal relationship, no shared state, and no constituent overlap. The IFP for one interaction proceeds independently and simultaneously with the IFP for any other interaction over disjoint participant contexts. They are not two items in a queue. They are two independent executions.

The conflict resolution priority function uses unpredictable slot randomness—no party can predict the priority ranking before the slot begins, and no party can manipulate it. Even within the conflict resolution for a single ICS, the resolution is deterministic—the same inputs always produce the same winner. There is no privileged position from which to extract value. The economic attack pattern known as MEV has no referent in a CC-compliant system: MEV arises from reordering transactions within a global sequence to extract value from position. There is no global sequence.

6.8 What IFP Specifies and What Implementing Networks Decide

Normative requirements — every CC-compliant IFP must satisfy:

A CC-compliant IFP must implement all four phases: Preparation, Formation, Confirmation, Commit. The certification threshold is $2f+1$ of $3f+1$ constituents. The commit

rule is $\text{Pos}(X) \geq 2f{+}1$ AND $\forall Y \neq X : \text{Neg}(Y) \geq f{+}1$. The conflict resolution priority function must be deterministic and unpredictable in advance. The Tesseract must be produced atomically—present in all participants' context chains named in P_ι, or in none. Confirmation state—cur_v, Pos, Neg, status—must persist across views. The monotonic climb is never reset by a view timeout. Safety and liveness must hold under partial synchrony with at most f Byzantine constituents.

Implementing network choices — what CC6 specifies for MOI:

How messages are routed within the ICS—whether a coordinator assists or constituents gossip directly. The specific mechanism for deriving slot randomness R. How the bound witness set and any additional witnesses are selected beyond what is embedded in $\text{Trust}(p)(p)$. The concrete Tesseract embedding in the MDAG. Timing parameters: slot duration, view length, timeout thresholds.

The leaderless monotonic confirmation protocol specified in Section 6 is the canonical form of a CC-compliant IFP. Implementing networks may use coordination-assisted execution as a stepping stone toward this model, provided safety and liveness are preserved at each stage. The progression from coordination-assisted to fully leaderless execution is a CC6 specification matter.

The invariant across all implementing network choices: the value model is constant, ordering does not exist at the protocol level, finality is deterministic and local, and the Tesseract is the atomic unit of finality.

6.9 Implications

The Interaction Finality Protocol constitutes a new finality model: hyper-local, immediate, and globally verifiable.

Multiple interaction types in parallel. Different participants interacting simultaneously execute in full parallel. Alice and Bob's interaction, Carol and Dave's interaction—their ICS sets are disjoint. No ordering across them is required or computed.

Same participant, different contexts, in parallel. Alice's interaction in her professional context and her interaction in her personal context execute in parallel if the ICS sets are disjoint. A participant is not a serialisation point. Their contexts are.

Multiple participants in a shared space, in parallel. Within a shared inherited context—a market, a governance space—interactions between different participant pairs execute in parallel. Only interactions naming the same participant at the same context height require ordering relative to each other. The shared space does not serialise. The conflict set does.

Immediate finality — update once, always fresh. Once the constituent set commits, the Tesseract is final. No probabilistic waiting, no confirmation count, no fork resolution. The context is updated exactly once and permanently. There is no stale state problem: context state is only written at Tesseract commit, and once written it does not change.

Hyper-local finality, global connectivity. Finality is local to the ICS—the $3f{+}1$ witnesses of that specific interaction. The Tesseract commit proof is globally verifiable by any node without re-running the agreement. Local agreement, global auditability. The network is connected without being coupled.

KYC and compliance across boundaries. A participant's KYC attestation is carried in Trust(p) and finalised through IFP. When that participant moves to a new institution or jurisdiction, the attestation travels with them in their Context Superstate. The new institution verifies the existing Tesseract-committed attestation rather than re-running KYC from scratch. Compliance is participant-anchored, not institution-anchored. Cross-boundary compliance without centralised databases.

Observation

Finality has always been local. When Alice hands Bob a banknote, the transaction is final the moment Bob accepts it. No third party confirms. No global state is updated. Two participants, one moment, one agreement.

But the banknote example understates the point. Real value interactions are not always bilateral. They involve witnesses—parties with contextual standing to attest that the transaction occurred correctly. What CP and IFP specify is that those witnesses must be *contextually relevant*: participants whose relationship to Alice and Bob gives their attestation actual informational weight.

Today's distributed systems get this exactly backwards. A bank transfer between two institutions in New York is witnessed by validators in Singapore, Seoul, and Frankfurt who have no relationship to either party, no context for the transaction, and no stake in its correctness beyond their general protocol obligation. They attest to events they are not in context of. This adds latency—messages must travel to parties with no proximity. It adds bottleneck—every validator witnesses every transaction, so the witness set scales with the network, not with the interaction. And critically, it adds no trust. Trust comes from witnesses who *are in context*: who know the participants, who have interacted with them, whose standing is anchored in a shared history. A Singapore validator witnessing a New York transfer brings none of that. Their attestation is a cryptographic formality, not a trust contribution.

CP's witness embeddedness in Trust(p) is the structural answer. The witness set is participant-constituted and interaction-specific. Alice's witnesses know Alice. Bob's witnesses know Bob. The intersection is the constituent set—a group with genuine contextual standing to attest. IFP finalises the transition within that set. The global network is not excluded from verification; it is simply not required for agreement. Anyone can later verify the Tesseract. But the agreement itself is reached by those who are actually in context.

This is what IFP formalises. The constituent set is the agreement primitive. The Tesseract is the settled fact. The global network is the verification layer. These are three different things. Classical distributed systems collapsed them into one. IFP restores the distinction.

7 Context Power

7.1 The Semantic Contract

Context Power realises two theoretical objects from the Contextual Compute series on a live network.

The first is Emergent Participant Intelligence. CC3 establishes that as a participant accumulates interactions, the metric tensor $g_p(t)$ of their position on the participant manifold evolves continuously. $\mathrm{EPI}(P, t) = g_p(t)$ is the formal statement that a participant's computational intelligence—their accumulated context—is a geometric quantity shaped by the full history of their interactions. It is participant-specific, continuous, and irreversible. It cannot be purchased, transferred, or manufactured. It can only be earned through interaction.

The second is $\mathrm{Trust}(p)(p)$. CC4 establishes $\mathrm{Trust}(p)(p)$ as the structural anchor of the Context Superstate—the dimension that gives the other six sub-contexts their participant-indexed character. $\mathrm{Trust}(p)$ accumulates through witnessed interactions. It carries the witness set $W(p)$, the inherited contexts, the Trust Quotient, and the Reputation Index. It is the operational form of the metric tensor: what $g_p(t)$ is geometrically, $\mathrm{Trust}(p)$ is computationally.

Context Power is the measure that makes both quantities computable, comparable, and usable for witness selection and protocol governance. It answers a question that neither EPI nor $\mathrm{Trust}(p)$ alone can answer: given two participants with different interaction histories, different identity attestations, and different capital commitments—how much influence does each carry within the protocol?

The answer must satisfy a constraint that follows directly from the K-Machine's structure: trust is a flow, not a stock. The participant dimension—WHO—is always present and always evolving. Trust that can be accumulated once, stored indefinitely, transferred to another participant, or reused across protocols is not participant-indexed trust. It is information about trust, stored in the wrong category. Context Power is designed to be structurally incapable of these operations—not by policy enforcement, but by the same categorical argument that makes value non-copyable.

7.2 Trust as Flow: The Ontological Foundation

Existing security models treat trust as a stock. Capital is committed at a point in time. The commitment earns influence proportional to its size. Influence persists as long as capital is committed. Capital can be delegated, sold, or reused across protocols. Security scales with the total committed capital.

This model is categorically correct for information. Capital is a Cartesian object—it can be accumulated, stored, transferred, and reused. Applying it to trust produces the same category error as applying information primitives to value: the diagonal morphism is present in the substrate, and all the consequences of copying follow.

Definition 7.1 (Trust as Stock)**.** In a capital-centric model, trust is a conserved quantity that can be accumulated through capital commitment, stored indefinitely, transferred

through delegation or liquid staking, and reused across protocols. Trust at time t determines influence at time $t+1$.

The consequences follow with necessity. Capital earns yield proportional to stake. Larger stakeholders earn more absolute yield. Yield is reinvested, compounding advantage. Economies of scale favour large operators. Capital concentration increases monotonically. The trajectory is centralisation—not as a failure mode but as the mathematical consequence of the model.

Definition 7.2 (Trust as Flow)**.** In Context Power, trust is a non-conserved quantity that must be generated through continuous participation, maintained through ongoing demonstration, bound to the participant's identity and interaction history, and verified per context. Trust at time t does not determine trust at time $t+1$.

The shift from stock to flow is the operational form of the K-Machine's structure. In the K-Machine, the participant's metric tensor $g_p(t)$ evolves with every interaction and does not persist unchanged through inactivity. A participant who stops interacting does not retain the metric tensor geometry they accumulated. The geometry changes—geodesics lengthen, distances grow. This is the geometric form of forgetting. Context Power implements the same principle: past contribution informs current influence through bounded-memory decay, but does not grant future influence independently of continued participation.

Theorem 7.3 (Incentive Structure Determines Centralisation Trajectory)**.** *The long-term character of a consensus system is determined by what resource naturally consolidates under its incentive structure. In capital-centric models, capital consolidates—that is centralisation. In Context Power, demonstrated participation consolidates—that is decentralisation. The trajectory is opposite.*

7.3 The Three Components

Definition 7.4 (Context Power)**.** Context Power Π_e^v for participant v in epoch e is:

$$\Pi_e^v = E_e^v + B_e^v + S_e^v$$

where E_e^v is the Economic score, B_e^v is the Behavioral score, and S_e^v is the Social score.

The three components are not interchangeable. Each addresses a distinct dimension of what it means for a participant to be trustworthy within the protocol. No component alone is sufficient. The gating function—specified in Section 7.4—ensures that weakness in any one component reduces total Context Power regardless of strength in the others.

7.3.1 Economic Score — Capital Commitment

The Economic score represents the participant's committed capital—their economic skin-in-the-game. Unlike proof-of-stake where capital commitment is the sole determinant of influence, in Context Power the Economic score is necessary but radically insufficient. It is one dimension of three.

The Economic score is the only component that participants can update directly—by increasing or decreasing their stake. It is also the only component subject to slashing: malicious participants can have their Economic score reduced as a penalty mechanism, providing economic alignment toward honest behaviour.

Two structural properties distinguish the Economic score from its counterpart in capital-centric models. First, the maximum Economic score a participant can hold is dynamically capped—as the network grows and Behavioral and Social scores accumulate, the economic cap decreases, preventing late-stage concentration by wealthy participants. Second, the relationship between capital and influence is concave—doubling stake does not double influence. Marginal returns from additional capital decrease with scale.

7.3.2 Behavioral Score — Demonstrated Participation

The Behavioral score reflects what the participant has actually done within the protocol—interactions validated, commitments honoured, protocol rules followed. It is earned through work, not purchased with capital. It cannot be transferred, because it is indexed to the participant's interaction history. A participant who acquires another participant's Behavioral score would acquire their history—which is constitutively impossible.

The Behavioral score realises the behavioral dimension of $\mathrm{EPI}(P, t)$. Each interaction the participant witnesses or initiates updates their computational context. The Behavioral score is the protocol's operational measure of that accumulated context, computable from the interaction record.

Decay is a structural property of the Behavioral score. Scores earned in past epochs are removed after a bounded window and redistributed as rewards for current activity. This is not a penalty mechanism. It is the operational form of the K-Machine's geometry: geodesics that are not traversed lengthen. A participant who stops participating does not retain the Behavioral score they accumulated. They must continuously demonstrate participation to maintain influence. This is behavioural decentralisation—influence is earned continuously, not accumulated once.

7.3.3 Social Score — Identity and Network Contribution

The Social score has two sub-components: identity and geo-distribution. Together they address a question that neither Economic nor Behavioral scores can answer: is this a unique, verifiable participant contributing to network diversity—or a manufactured presence?

The identity problem: WHO, not what. Existing approaches to digital identity solve the wrong problem. CAPTCHA and similar mechanisms solve the species problem—proving that an entity belongs to the category "human." This is a WHAT question. It does not answer the WHO question: is this the same participant who interacted before, and with what confidence?

Proof of humanness $\neq$ proof of continuity $\neq$ proof of identity.

A wallet is a Cartesian object. It holds credentials, tokens, and keys. It can be copied, transferred, and stolen. When the self-sovereign identity movement says "your identity lives in your wallet," it makes the same categorical error as treating value like

information—turning a relational, participant-dependent phenomenon into a static, copyable artefact. Someone steals your wallet. Do they become you? Obviously not. The identity was never in the wallet.

Participant recognition requires three irreducible components. Any two without the third is spoofable. Credential (WHAT) and Behavior (HOW) without Human element (WHO) is a sophisticated automated agent. Credential and Human element without Behavior is a first-time stranger with documents. Behavior and Human element without Credential is someone recognisable but not formally verifiable. The Behavioral score already carries the HOW dimension, so the Social score focuses on the credential and human dimensions.

WHO recognition is community-delegated. The protocol does not know who you are. Your community knows who you are. The CP protocol provides infrastructure for communities to express recognition. What counts as valid recognition—which identity providers are trusted, what attestation types are required, what confidence threshold is sufficient—is determined by the community governing the identity standard through Context Power-weighted governance. A village cooperative and a sovereign wealth fund have radically different recognition requirements. The protocol has no business imposing a universal standard on contexts that are fundamentally different. The protocol triangulates across multiple identity standards as chosen by the community deploying it. It does not define WHO you are. It provides the computational substrate within which your community's recognition of you can be expressed and verified.

Self-sovereign identity storage. Identity attestations are stored in the participant's own Context Superstate—specifically in Trust(p)—not in a central identity registry. There is no central registry to compromise. Each participant carries their own proofs. The network validates attestations against the identity standard at consensus time. This is self-sovereign by construction, not by policy.

The two-tier model. Identity operates at two levels. The identity standard is the minimum requirement for protocol admission—a binary gate. The identity sub-component of the Social score measures the strength of attestations above this minimum—a continuous measure. Participants who invest in stronger, more diverse, more frequently renewed attestations carry higher Social scores and therefore higher Context Power.

Identity score. The identity sub-component ι_e^v measures the strength, diversity, and freshness of a participant's verified identity attestations. Attestations are issued by decentralised identity providers approved through community governance. Different attestation types carry different weights, freshness half-lives, and maximum counts—all parameters governed by the community. When an identity marker is shared across multiple participants, the score contribution of that marker is divided with a penalty exponent, discouraging identity sharing. The maximum number of protocol validators that a single individual may operate is bounded and enforced through the identity attestation mechanism.

Identity, like Context Power itself, is a flow. Attestations expire and must be renewed. Revocation is both automatic—when a provider publishes a cryptographic revocation—and governance-initiated, when fraud is suspected but not cryptographically provable. A participant cannot establish identity once and retain it indefinitely.

Geo-distribution score. The geo-distribution sub-component ρ_e^v measures the participant's marginal contribution to network diversity—geographic, infrastructure, and operational. A participant in an underrepresented region on uncommon infrastructure receives

a high geo-distribution score. A participant adding to an already dense concentration receives a score approaching zero. The scoring is endogenous: concentration in any category automatically reduces the marginal reward for joining that category. The protocol rewards diversity without requiring central coordination of placement.

The combined social contribution for participant v in epoch e is:

$$\Phi_e^v = \iota_e^v + \rho_e^v$$

Social scores are non-transferable—identity attestations are bound to the participant's cryptographic identity, and geo-distribution reflects where the participant actually operates. Neither can be manufactured by capital commitment alone.

Proof of Humanity—forward reference. Static identity verification proves a human registered. Proof of Humanity proves a human is present at consensus time—not just at enrolment. Planned for H2 2026, when implemented it will be incorporated into the identity sub-component as an additional attestation type within the existing framework.

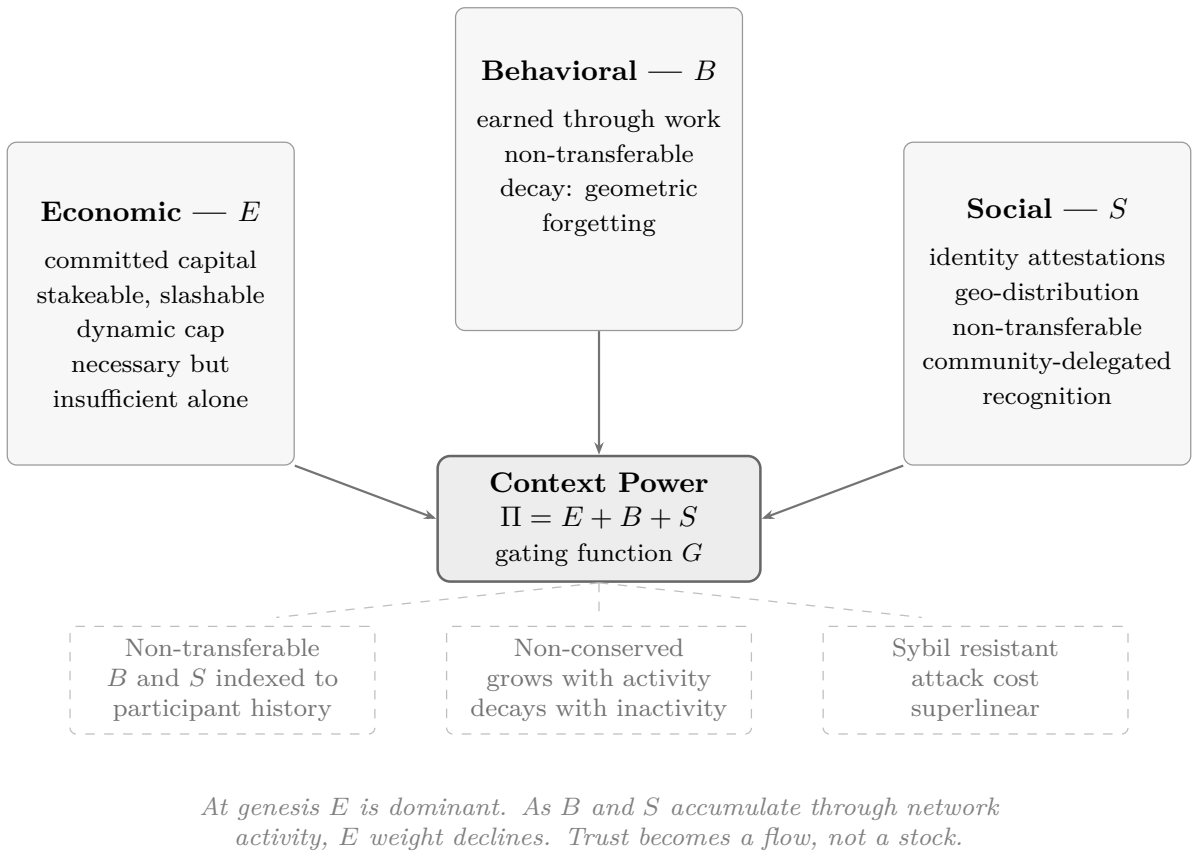

Figure 8: Context Power three-component structure. E (Economic) is the only component participants can update directly and is subject to slashing. B (Behavioral) is earned through witnessed interactions and decays over a bounded window. S (Social) combines identity attestation strength and geo-distribution contribution. The multiplicative gating function G^v ensures all three dimensions must be present—weakness in any one reduces total Context Power regardless of strength in the others.

7.4 Structural Properties

Definition 7.5 (Complete Context Power Formula)**.**

$$\Pi_e^v = \Pi_{\text{raw}}^v \times G^v$$

where $\Pi_{\text{raw}}^v = w_E \cdot E_e^v + w_B \cdot B_e^v + w_S \cdot S_e^v$ and G^v is a multiplicative gating function over all three components.

The gating function ensures no component can be neglected. A participant with strong Economic and Behavioral scores but negligible Social score has their total Context Power reduced by the gate—regardless of the absolute values of the strong components.

Theorem 7.6 (Non-Conservation)**.** $\sum_v \Pi_e^v$ *varies with* e*—Context Power is non-conserved. There is no fixed total that is redistributed among participants. It grows with network activity and decays with inactivity.*

Proof. Behavioral rewards scale with interactions executed. Social rewards scale with new participants joining and with geo-distribution activity. Neither is fixed. As network activity grows, total Context Power grows. As participants become inactive, decay reduces their scores without transferring them in full to others. Therefore $\sum_v \Pi_e^v$ is not constant across epochs. □

Theorem 7.7 (Non-Transferability)**.** *There exists no operation by which Context Power can be transferred from participant* v *to participant* u*.*

Proof. The Behavioral score B_e^v is indexed to v's interaction history—the sequence of interactions v has witnessed and executed. No operation transfers this history to u, because u's history is constitutively distinct from v's. The Social score S_e^v is indexed to v's identity attestations and geographic location—properties of v as a participant. These cannot be reassigned to u without changing who u is. The Economic score E_e^v can be unstaked by v, but unstaked capital is not transferred to u's Economic score—it exits the staked pool. Therefore no component of Π_e^v can be transferred. □

Theorem 7.8 (Restaking Impossibility)**.** *Context Power cannot be restaked across protocols—not by policy prohibition, but because there is no conserved quantity to restake.*

Proof. Restaking requires a quantity Q committed to protocol A to be simultaneously committed to protocol B. By Theorem 7.6, Context Power is non-conserved: it has no fixed quantity independent of the context in which it was earned. By Theorem 7.7, it cannot be transferred. Therefore the operation of committing the same Context Power to two protocols simultaneously is undefined—not prohibited, but absent from the grammar. □

7.5 The Decay Mechanism

Decay is the operational form of the K-Machine's metric tensor evolution through non-interaction. In the K-Machine, geodesics that are not traversed lengthen—the geometric form of forgetting. In Context Power, Behavioral and Social scores earned in epoch e are removed after a bounded decay window ω and redistributed as rewards for current epoch activity.

Definition 7.9 (Decay)**.** For participant v in epoch e, the score decayed is the score earned as reward in epoch $e - \omega$:

$$\delta B_e^v = \Delta B_{e-\omega_B}^v$$
$$\delta S_e^v = \Delta S_{e-\omega_S}^v$$

The updated scores are:

$$B_{e+1}^v = B_e^v - \delta B_e^v + \Delta B_e^v$$
$$S_{e+1}^v = S_e^v - \delta S_e^v + \Delta S_e^v$$

The decay mechanism has three structural consequences. First, it prevents early participants from maintaining permanent advantage—historical contribution erodes unless reinforced by current participation. Second, it creates a bounded-memory relationship between past and present. Third, it continuously creates space for new participants to achieve competitive Context Power within a bounded number of epochs—structural fairness, not policy aspiration.

Definition 7.10 (Bootstrap)**.** At genesis, Behavioral and Social scores begin at their default initial state—no interaction history exists, so no behavioral or social rewards have been earned yet. The Economic score carries disproportionate weight early—a dynamic maximum weight ceiling $w_E^{\max}$ that decays toward a floor w_E^{floor} as Behavioral and Social scores accumulate through network activity. The transition is automatic and deterministic— no governance intervention required. At steady state, the majority of Context Power derives from non-economic sources.

This bootstrap mechanism follows from the K-Machine's context accumulation model: at genesis, no interaction history exists, so the Behavioral and Social dimensions carry no earned contribution. As the network accumulates interactions, the geometric richness of the participant manifold grows. Context Power tracks this growth faithfully.

7.6 Sybil Resistance

Context Power makes Sybil attack superlinear in cost—not through a single gate, but through the interaction of all three components simultaneously.

Theorem 7.11 (Capital Insufficiency)**.** *Let adversary A possess capital $C \to \infty$. Under Context Power:*

$$\lim_{C\to\infty} \text{Influence}(A) = m \cdot c \cdot \varepsilon$$

where m is the number of valid identities A can acquire, c is the maximum participants per identity, and ε is the maximum Context Power per participant. Under capital-centric models, $\text{Influence}(A) \propto C$*—linear and unbounded.*

An adversary attempting a Sybil attack must: acquire valid identity attestations for each additional participant—scarce and independently verified; operate each participant from a distinct geographic and infrastructure configuration; maintain continuous Behavioral participation for each participant—Behavioral history cannot be manufactured; and remain within the dynamic Economic cap, which decreases as the network grows. The cost of attack scales superlinearly with the number of Sybil identities. There is no shortcut through capital alone—the Behavioral and Social dimensions block it structurally.

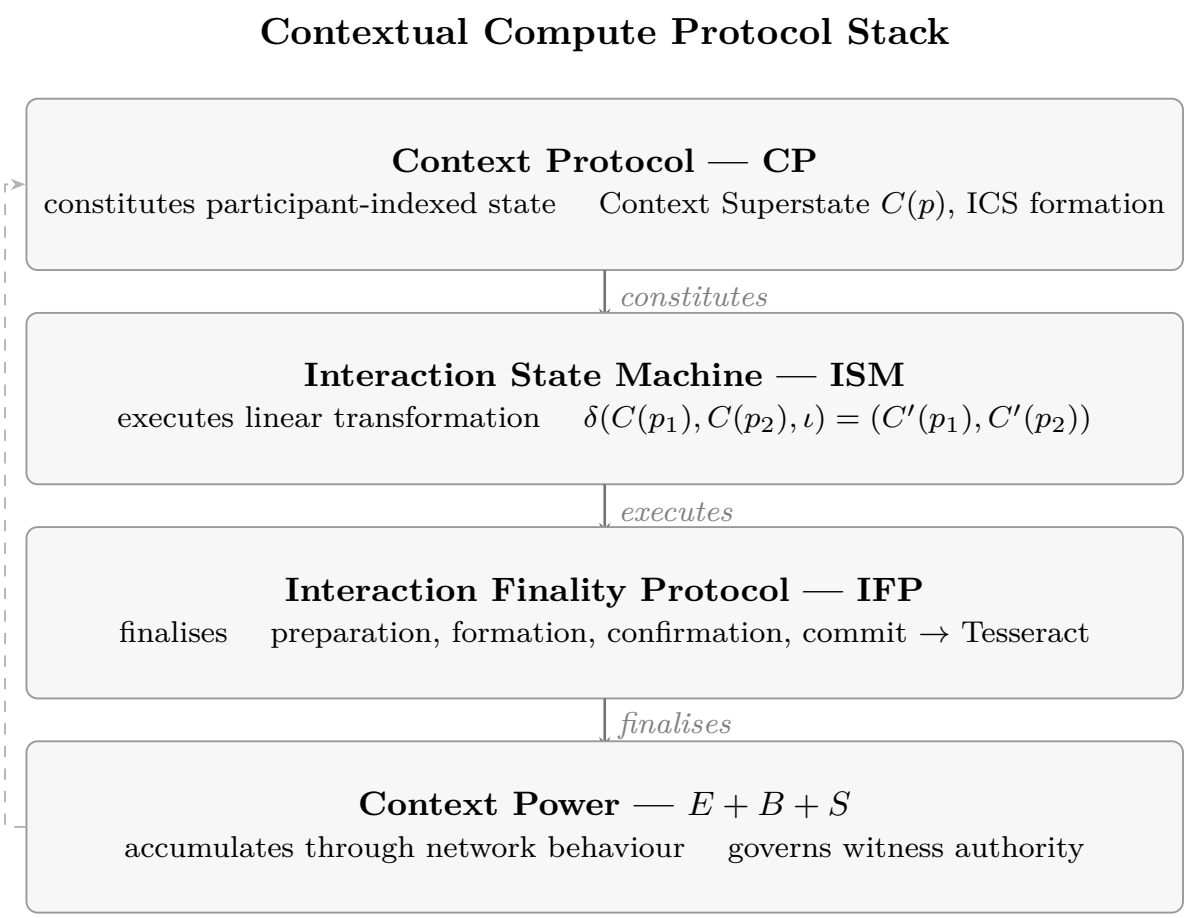

Figure 9: Contextual Compute Protocol Stack. CP constitutes participant-indexed state. ISM executes linear transformations over the ICS. IFP achieves deterministic local finality. Context Power governs witness authority and accumulates from completed Tesseracts. The four components are a fixed sequential derivation: each is the precondition for the next. The Tesseract (bottom) is the cross-stack record that simultaneously embeds outputs from all four components as a single immutable object.

7.7 Context Power and Witness Authority

Context Power governs which participants are eligible to serve as witnesses within the IFP and with what weight. A participant is eligible to serve as a witness if their Context Power meets the minimum threshold:

$$\Pi_e^v \geq \Pi_{\min}$$

This is not a capital threshold. It is a multi-dimensional commitment threshold spanning Economic alignment, demonstrated Behavioral participation, and Social verifiability. A participant with unlimited capital but no Behavioral history or identity attestation does not meet it.

Within the IFP, selection probability scales with Context Power—participants with higher Context Power are more likely to be selected—but selection is probabilistic, ensuring that no single participant can guarantee selection regardless of their Context Power level.

This is the protocol's realisation of the K-Machine's fourth structural consequence: se-

curity from participation. The interaction mesh grows more secure as more participants accumulate richer context. Security is a property of the network's interaction history, not of committed resources. It cannot be purchased. It can only be earned.

7.8 Compliance Requirements for Context Power

A CC-compliant Context Power implementation must satisfy the following normative requirements. The three-component structure—Economic, Behavioral, Social—is normative. Non-transferability of Behavioral and Social scores is normative. Decay is normative for Behavioral and Social scores. The dynamic Economic cap is normative. The bootstrap mechanism is normative. The minimum Context Power threshold for witness eligibility is normative—a multi-dimensional threshold, not a single-dimension capital gate.

Specific epoch duration, reward rate formulas, decay window values, component weight evolution schedules, identity provider governance, and geo-distribution scoring parameters are implementing network choices documented in CC6.

7.9 Implications

Context Power constitutes a new model of network authority—growing, distributed, and earned through participation.

Growing decentralisation. As B and S accumulate through network activity, the E weight in $\Pi = E + B + S$ declines toward its floor. Resources that were consolidated at genesis disperse over time as Behavioral and Social influence grows across a wider participant base. This is structural decentralisation—not a governance decision, but a consequence of the mathematics.

Structural decentralisation through participation. Context Power is earned through participation. Every interaction contributes to B. Every identity attestation and geographic distribution strengthens S. The network makes decentralisation rewarding in a way that directly measures actual contribution, not token holdings.

The real Nakamoto coefficient. Classical TPS is a raw throughput number. It says nothing about the trust distribution of the witnesses generating that throughput. A network with high TPS but all witnesses controlled by three entities in a single region has a different trust-adjusted throughput than one with lower TPS and genuinely distributed Context Power across people and geography. Context Power's Social component S—which includes both identity attestations and geo-distribution scoring—makes this measurable. The Nakamoto coefficient in a CC-compliant network is a function of Π across the witness population, measuring both human distribution and geographic distribution. Not a count of entities. A function of earned trust.

Holistic reward. $\Pi_e^v = E_e^v + B_e^v + S_e^v$ rewards the whole participant—their economic commitment, their interaction history, and their social contribution. Not just stake. Not just activity. The participant as a whole.

Observation

In every prior consensus system, influence is purchased. Capital buys stake. Stake buys votes. Votes buy outcomes. The accumulation curve is monotone: more capital always means more influence, and capital compounds.

Context Power inverts this. Influence is earned—through interaction, through identity, through presence. It decays when not renewed. It cannot be transferred to someone who has not earned it. It cannot be concentrated without limit because the dynamic cap closes with network maturity. The network becomes more decentralised as it grows, not less. This is not a design aspiration. It is a mathematical consequence of treating trust as a flow indexed to the participant, not a stock that can be accumulated and held.

The K-Machine has always described this. $\mathrm{EPI}(P, t) = g_p(t)$ says that a participant's computational influence is a geometric quantity that evolves continuously with their interaction history. Context Power is that quantity made operational. The formalism is pointing at something that was always true about how trust accumulates in human systems. The computation is now aligned with it.

8 Protocol Composition and Compliance

8.1 How the Four Components Compose

The four components of the Contextual Compute Protocol are not independent modules. They are a single derivation chain in which each component is the precondition for the next.

CP constitutes the participant. It defines what a participant is—the 4-tuple $p = (\mathrm{id}, \sigma, f_p, H)$, the Context Superstate $C(p) = \mathrm{Trust}(p) \triangleright \{\mathrm{Assets}, \mathrm{Storage}, \mathrm{Logic}, \mathrm{Keys}, \mathrm{Prefs}, \mathrm{Intel}\}$, the context chain that makes succession immutable, the inherited contexts that make shared computation possible without shared state. CP is the substrate. Without it, there is no participant-indexed state for any other component to act on.

ISM executes the transformation. Given a valid ICS formed by CP over named participant contexts, ISM applies the linear transformation f_ι—drawn from $\mathrm{Logic}(C_i)$ of the relevant execution context—to compute the post-contexts. ISM enforces the two-bucket separation: value objects in the value bucket are linear and participant-bound; information in the information bucket is Cartesian and freely readable. ISM produces the post-contexts that IFP will finalise.

IFP finalises the transition. Given the post-contexts produced by ISM, IFP runs the monotonic elimination protocol over the constituent set $\mathcal{C}_\iota = P_\iota \cup W_\iota$ until the $2f{+}1$ threshold is reached. When the commit rule is satisfied, the pre-contexts are consumed, the post-contexts are produced, and the Tesseract is crystallised. Finality is constituted at this moment.

Context Power governs who may participate as witnesses throughout. It determines selection probability for the witness set $W(\iota)$ embedded in $\mathrm{Trust}(p)(p)$, the weight each witness carries within IFP, and the accumulation of trust that each completed Tesseract contributes to the participants involved.

The composition is therefore:

$$\text{CP constitutes} \longrightarrow \text{ISM executes} \longrightarrow \text{IFP finalises} \longrightarrow \text{Context Power accumulates}$$

Each arrow is forced. No component can be reordered. No component can be removed without collapsing the model.

8.2 The Tesseract as the Cross-Stack Record

The Tesseract is the record that spans all four components simultaneously. It is the unit of composition.

From CP: the Tesseract records the participant set P_ι and their updated contexts at height $h{+}1$—the horizontal link connecting each participant's context chain to every other participant's chain at the moment of their interaction.

From ISM: the Tesseract records the state transition δ—the typed domain operation that was executed, verifiable against the Logic of the inherited context.

From IFP: the Tesseract carries the commit proof Π—the $2f{+}1$ constituent attestations that constitute finality, verifiable by any party from the witness set alone.

From Context Power: the Tesseract is the interaction event from which Behavioral scores accumulate for each participating witness, contributing to the metric tensor evolution of every constituent.

$$\mathcal{T}(\iota) = \big(\mathrm{id}_\iota,\ P_\iota,\ \delta,\ \{(p, v_p) : p \in P_\iota\},\ \Pi\big)$$

This single object is simultaneously a CP record, an ISM record, an IFP record, and a Context Power record. It is the crystallisation of all four K-Machine dimensions—WHAT, WHERE, HOW, WHO—as one immutable fact.

8.3 Normative Compliance Requirements

8.3.1 Minimum CC-Compliant Implementation

Context Protocol. Single context vector per participant at any height. Anchor chain $\alpha(h{+}1) = H(\mathrm{id}(p)\|h{+}1\|\alpha(h)\|\sigma(h{+}1))$ is normative. Witness set $W(p)$ embedded in $\mathrm{Trust}(p)$ and committed in the context chain. Participant sovereignty: no context transition without cryptographic authorisation from its owner. Value is participant-indexed: Assets, Storage, Keys, and Intel exist only within the $C(p)$ of the participant they belong to. An inherited context C_i is itself a participant with its own sovereign $C(C_i)$—it may hold its own Assets, Storage, Logic, Keys, Prefs, and Intel as a participant in its own right. What C_i may not do is hold value-bearing state on behalf of other participants. $\mathrm{Assets}(p)$ for any participant p inhabiting C_i must remain in $C(p)$. This separation is structural: the grammar of the execution environment does not admit transferring a participant's assets into a shared context object. Context reset mechanism preserves linearity: participant frozen at current valid height from moment of reset initiation.

Interaction State Machine. State transition function scoped to named participant contexts only. Two-bucket model enforced: value bucket is linear, information bucket is Cartesian. Domain model includes participant, asset, and logic domains with an extension mechanism. Re-entrancy non-expressible at the type level. Atomicity, linearity, locality, and determinism hold for every execution.

Interaction Finality Protocol. ICS fully determined before execution begins. Preparation threshold $2f{+}1$ of $3f{+}1$. Conflict resolution priority function deterministic and unpredictable in advance. Commit rule: $\mathrm{Pos}(X) \geq 2f{+}1$ AND $\forall Y \neq X : \mathrm{Neg}(Y) \geq f{+}1$. Confirmation state persists across views—never reset. Tesseract produced atomically across all participant context chains named in P_ι. Pre-contexts consumed by a committed interaction are not referenceable by any subsequent interaction. Safety and liveness under partial synchrony with at most f Byzantine constituents.

Context Power. Three-component structure: Economic, Behavioral, Social. Non-transferability of Behavioral and Social scores normative. Decay mechanism normative. Dynamic Economic cap normative. Bootstrap mechanism normative. Minimum Context Power threshold for witness eligibility is multi-dimensional.

8.3.2 What Implementing Networks May Vary

The specific context storage data structure. The witness selection mechanism beyond the bound witness set. The programming language and execution environment for ISM logic. The slot randomness derivation mechanism. The IFP coordination mechanism (operator-assisted or fully leaderless), provided safety and liveness are preserved. Epoch duration, decay windows, reward rates, component weight schedules, identity governance procedures, and geo-distribution scoring parameters. The specific hash function in the context chain. The specific f for a given interaction—the $2f+1$ of $3f+1$ structure is normative; f itself is an implementing network or participant choice governed by Φ.

8.3.3 What Implementing Networks Must Not Vary

Participant sovereignty. No context transition without cryptographic authorisation from the context owner—Consent (Axiom 7.2, CC4) made operational.

Value linearity. No diagonal morphism on value objects. Absent from the grammar, not enforced by policy.

Witness embeddedness. Witnesses must be embedded in Trust(p) and committed in the context chain. A runtime-assigned external witness pool is not CC-compliant.

Shared compute without shared state. Inherited contexts carry Logic and Prefs as their computation contribution. A system in which participants deposit value into a shared context object—making that context a custodian of other participants' assets—is not CC-compliant for those value objects.

ICS pre-determination. Runtime discovery of participants or witnesses violates Context Boundedness (Theorem 6.3).

Tesseract atomicity. The Tesseract must appear in the context chain of every participant named in P_ι, or in none. Partial commitment across the named participant set is not permitted. A single-participant interaction is valid—its Tesseract appears only in that participant's chain. A multi-participant interaction naming $p_1, \ldots, p_n$ must commit to all n chains simultaneously.

Context Power multi-dimensionality. A single-capital-criterion system is not CC-compliant regardless of throughput or finality speed.

8.4 The Derivation Chain

Every normative requirement in CC5 derives from a specific source in CC2, CC3, or CC4. Nothing is a design choice. Table 4 makes this explicit.

Table 4: Every normative requirement traced to its theoretical source.

CC5 Normative Requirement	**Source**
Participant sovereignty	CC4 Axiom 7.2 (Consent); CC4 Property 4.3
Value linearity	CC2 Domain Separation Theorem; CC4 Axiom 7.3
Context chain structure	CC3 KCC trajectory τ_p; CC4 Section 5.6
Shared compute without shared state	CC4 Axiom 7.6 (Commutativity)
ICS pre-determination	CC4 Definition 5.7; CC4 Section 5.13
Two-bucket model	CC2 Cartesian Degeneracy; CC4 domain separation
Preparation threshold $2f+1$ of $3f+1$	CC4 Axiom 7.4 (Witness Sufficiency)
Confirmation state view-persistent	CC4 Theorem 5.11 (Value–Agreement Equivalence)
Tesseract atomicity	CC4 Axiom 7.3 (Linearity); CC4 Corollary 5.12
Context Power non-transferability	CC3 EPI non-transferability; CC4 Intel(p) anchoring
Context Power decay	CC3 metric tensor evolution $g_p(t)$
Context Power multi-dimensionality	CC3 Proposition 7.1 (Security from Participation)
Re-entrancy non-expressible	CC4 Axiom 7.3
Witness embeddedness in Trust(p)	CC4 Section 5.22

9 Bridge to MOI: CC Protocol Objects and Their Realisations

9.1 The Relationship Between CC5 and CC6/CC7

CC5 is a normative specification. It defines what any CC-compliant system must be. CC6 and CC7 specify MOI—the first CC-compliant open network and the live reference implementation of Contextual Compute.

The relationship is not one of approximation. MOI does not approximately implement the CC Protocol. It precisely implements it, with specific choices made within the degrees of freedom that CC5 leaves open, and with a progressive release roadmap that moves from coordination-assisted execution toward the fully leaderless model that CC5 specifies as canonical.

Every CC Protocol object defined in CC5 has a corresponding MOI realisation. Every MOI engineering choice that goes beyond what CC5 mandates is documented in CC6 or CC7 as a decision made within the space CC5 permits. This section maps CC Protocol objects to their MOI realisations. It is a bridge document—not a specification of MOI, but a map that shows where CC5 ends and CC6 begins.

9.2 CC Protocol Object to MOI Realisation Map

The participant 4-tuple $p = (\mathrm{id}, \sigma, f_p, H)$. MOI realises this as a protocol-native participant context anchored to the genesis context. The participant identifier id is derived from the participant's public key. f_p supports both classical and post-quantum signature schemes, selectable per participant through $\mathrm{Prefs}(p)$. H is the cryptographic hash function used in the context chain, specified in CC6.

The Context Superstate $C(p) = \mathrm{Trust}(p) \triangleright \{\mathrm{Assets}, \mathrm{Storage}, \mathrm{Logic}, \mathrm{Keys}, \mathrm{Prefs}, \mathrm{Intel}\}$. MOI realises this as a participant context stored and maintained by the participant's witness set across the MDAG. The Merkle root is the canonical commitment. Sub-context content may be distributed across context nodes for availability; the root is singular and final.

The context chain $\alpha(h{+}1) = H(\mathrm{id}(p) \| h{+}1 \| \alpha(h) \| \sigma(h{+}1))$. MOI implements this exactly as specified. The context chain is the participant's private blockchain—a self-attesting commitment to the complete succession history of their context, verifiable in $O(h)$ hash operations.

The interaction 5-tuple $\iota = (S, T, W, f_\iota, \mathrm{Auth}_\iota)$. MOI realises this through the ICS formation protocol specified in CC6, which adds the operator O, randomness R, and time slot T as coordination parameters beyond the CC5 normative minimum.

The ICS $= (P_\iota, C_\iota, \Phi)$. MOI realises this as $\mathrm{ICS}(\iota) = (O, P_\iota, C_\iota, R, T)$—extending the CC5 definition with operator, randomness, and slot parameters that support coordination-assisted execution. The O, R, T parameters are MOI-specific coordination efficiency additions that CC5 explicitly leaves to implementing networks.

The ISM state transition $\delta(C(p_1), \ldots, C(p_n), \iota) = (C'(p_1), \ldots, C'(p_n))$. MOI realises this through COCO—the Contextual Compute language—which provides the typed ex-

ecution environment in which f_ι is expressed. COCO implements the two-bucket model, the typed domain primitives, and re-entrancy elimination at the type level. The COCO language specification is CC6 material.

The monotonic elimination protocol—Algorithm 6.1. MOI realises this across three progressive releases. Release 1 is operator-assisted—a coordination-assisted stepping stone with the same safety and liveness properties, realised as Prepare, Propose, Vote, Commit, closer to 4Δ in practice. Release 2 reduces operator statefulness through broadcast batching and mempool certification. Release 3 is the full leaderless monotonic confirmation protocol as specified in Section 6. The value model, finality properties, and Tesseract structure are invariant across all three releases. The coordination mechanism evolves.

The Tesseract $\mathcal{T}(\iota) = (\mathrm{id}_\iota, P_\iota, \delta, \{(p, v_p) : p \in P_\iota\}, \Pi)$. CC5 defines the Tesseract as an abstract CC Protocol object. MOI realises this as a concrete data structure within the MDAG. The MDAG is a multi-dimensional mesh with two distinct edge types: transitive edges linking successive context states vertically within each participant's own chain, and interactive edges linking Tesseracts horizontally across different participants' chains. The Tesseract is the node at which both edge types meet—simultaneously advancing each participant's vertical context chain to height $h+1$ and creating the horizontal cross-reference that links all participating chains at that moment of interaction. The MDAG structure, Tesseract embedding, and traversal algorithms are CC6 material.

Context Power $\Pi_e^v = E_e^v + B_e^v + S_e^v$. MOI realises this as PoXt—Proof of Context—the live witness selection and reward mechanism operating across MOI's 2,100+ validator network. PoXt specifies the concrete parameter values: epoch duration, decay windows ω_B and ω_S, reward rates β and σ, economic cap formula, identity standard governance procedures, and the bMOI/sMOI token mechanics. These are CC6 and CC7 material.

9.3 What CC6 Specifies

CC6 is the MOI implementation specification. It documents:

The **MDAG**—the Multi-linked Composite DAG that stores participant contexts and Tesseracts. Its structure: transitive edges linking successive context states within a participant chain; interactive edges linking Tesseracts across participant chains. Its properties: acyclicity, storage locality (witnesses store only their contexts), deterministic finality through the Tesseract commit proof.

KRAMA—the full CP/IFP protocol stack as implemented in MOI. The three-release roadmap from operator-assisted coordination to fully leaderless monotonic elimination. The specific Prepare/Propose/Vote/Commit stages of Release 1. The ICS formation algorithm including operator selection, VRF-based randomness derivation, stochastic witness selection, and the bound/random witness composition.

COCO—the Contextual Compute language. The typed execution environment in which ISM logic is expressed in MOI. Its type system, domain primitives, and the mechanism by which re-entrancy is eliminated at the type level.

PoXt—the full Context Power implementation. Epoch mechanics, token structures (bMOI, sMOI), decay window parameters, reward rate formulas, economic cap dynamics, identity standard governance, and the long-term network dynamics analysis.

Context reset mechanics—the two-path recovery model: normal handoff when $2f+1$ context nodes are reachable, network-level reset at epoch boundary when they are not. The role of the System Context as the network's primordial anchor and recovery authority of last resort.

9.4 What CC7 Specifies

CC7 is the MOI economic and governance specification. It documents the token economics, validator incentive structures, governance mechanisms, and the long-term dynamics of the MOI network as a live CC-compliant system. CC7 is forthcoming from Sarva Labs.

9.5 The Live Reference Implementation

MOI is not a planned system. It is a live network with over 2,100 validators and an active developer and validator community. The CC Protocol objects defined in CC5 are operational in MOI today—at Release 1 of the KRAMA protocol stack, with Releases 2 and 3 on the roadmap.

The existence of MOI as a live network is significant for the CC Protocol specification in two ways.

First, it demonstrates that the normative requirements of CC5 are implementable. Every design decision in CC5—participant-indexed state, the two-bucket model, the ICS predetermination requirement, the Tesseract as the unit of finality, the three-component Context Power—has been implemented and operates under real network conditions. The specification is not theoretical conjecture. It is a distillation of what works.

Second, it establishes a compliance baseline. MOI is the first CC-compliant network. Future networks claiming CC-compliance can be evaluated against the same normative requirements that MOI satisfies. The CC Protocol specification is the standard. MOI is the existence proof.

10 Closing Statement

10.1 What This Paper Has Established

This paper has specified the Contextual Compute Protocol—the normative implementation standard for any system claiming to compute over value correctly. Four components have been defined, their derivations from the theoretical trilogy have been made explicit, and their composition into a single coherent protocol has been demonstrated.

The Context Protocol establishes the substrate: participant-indexed state in which $C(p)$ is the single authoritative cryptographic root of a participant's computational existence, succession is linear and irreversible, and shared computation never implies shared state. CP eliminates the condition that makes external consensus necessary—not by engineering a faster consensus mechanism, but by making the participant a computational primitive whose state is always locally known, always deterministically verifiable, and never anonymous.

The Interaction State Machine establishes the execution layer: a state transition function scoped exactly to the named participants of each interaction, enforcing the two-bucket separation between linear value and Cartesian information, making re-entrancy non-expressible at the type level, and producing deterministic post-contexts from deterministic inputs. ISM is not a restricted smart contract environment. It is computation structured around the correct category from the beginning.

The Interaction Finality Protocol establishes the agreement layer: a leaderless, deterministic, monotonic confirmation protocol over a bounded constituent set, producing finality in 2Δ after GST without global ordering, without a coordinator, and without probabilistic accumulation. IFP dissolves the distributed systems trilemma not by optimising within it but by demonstrating that the trilemma was an artefact of the wrong substrate—and that on the correct substrate, leaderlessness, determinism, and speed are simultaneously achievable.

Context Power establishes the trust layer: a three-component, non-transferable, decay-governed measure of participant influence that realises $\mathrm{EPI}(P, t) = g_p(t)$ from CC3 and $\mathrm{Trust}(p)$ from CC4 on a live network. Context Power makes trust a flow rather than a stock, decentralisation a structural consequence rather than a governance aspiration, and Sybil resistance a superlinear cost rather than a threshold to be purchased past.

The Tesseract unifies all four. It is the first computational object that encodes WHAT, WHERE, HOW, and WHO as a single irreducible record—simultaneously a CP record, an ISM record, an IFP record, and a Context Power record. It is the unit at which participant-indexed computation becomes final.

10.2 The Derivation Chain Is Complete

The Contextual Compute series now spans five papers forming a complete derivation chain from mathematical foundations to normative specification.

CC2 establishes that value and information are objects in categorically distinct monoidal categories. The diagonal morphism is absent from the value category. This is not a design preference. It is a theorem.

CC3 introduces the K-Machine—the abstract machine for value computation that places WHO as a primitive dimension alongside WHAT, WHERE, and HOW. It proves that anonymous global computation is recovered exactly at WHO $= 0$, establishing the Turing Machine as a special case of the K-Machine. It establishes $\mathrm{EPI}(P, t) = g_p(t)$ as the geometric form of participant trust accumulation.

CC4 derives the operational semantics of the K-Machine through PIC—six axioms that force the structure of value computation and define $\mathcal{V}_{\mathrm{PIC}}$ as a symmetric monoidal category without a diagonal morphism.

CC5—this paper—translates those axioms into a normative protocol specification. Every requirement derives from a specific theoretical source. Nothing is a design choice.

CC6 and CC7—forthcoming from Sarva Labs—specify MOI as the first CC-compliant open network, realising every CC Protocol object on a live system.

The chain runs from category theory through abstract machine theory through operational semantics through normative specification through live implementation. Each link is forced by the prior one. The protocol is not engineered. It is derived.

10.3 What Comes Next

Three questions are deliberately left open for CC6 and CC7.

The first is the KRAMA protocol stack in full—the complete specification of CP and IFP as implemented in MOI, including the three-release progression from operator-assisted coordination to fully leaderless monotonic elimination, the MDAG data structure, COCO language semantics, and the concrete ICS formation algorithm with VRF-based randomness and stochastic witness selection.

The second is the PoXt economic model in full—the complete specification of Context Power as implemented in MOI, including epoch mechanics, token structures, decay parameters, reward rate formulas, economic cap dynamics, identity standard governance, and the long-term network dynamics analysis.

The third is MOI as a system—its genesis context, its current network topology, its validator set composition, its live performance characteristics, and its roadmap from Release 1 through Release 3.

These are the correct content of a different document—one whose purpose is to specify a concrete network rather than to define the standard against which that network is measured.

10.4 The Claim

The Contextual Compute Protocol makes a claim that can be stated simply.

Every prior distributed system built on Cartesian foundations—regardless of its consensus mechanism, its throughput claims, or its parallelisation strategy—assumes the coordinator must be found. Either you trust one, or you elect one, or you assign one per slot, or you let committed resources decide. The coordinator is always external to the value. It must be constructed, trusted, elected, or computed at runtime. This generates the trilemma.

There is no third option within that substrate.

The CC Protocol is built on the recognition that the coordinator was never absent. Value cannot exist without the participant. The participant is the coordinator—not by assignment but by ontological necessity. When the machine is built for the correct category, coordination becomes local, finality becomes intrinsic, and the trilemma dissolves—not as an engineering achievement, but as a categorical consequence.

This is not a performance improvement over consensus. It is the elimination of the condition that makes external consensus necessary. What remains is linear agreement—scoped exactly to the participants and witnesses of each context, and nothing beyond them.

It was never about the ordering. It was always about the orderer.

A Notation and Symbol Index

Symbol	**Meaning**
p	Participant 4-tuple $(\mathrm{id}, \sigma, f_p, H)$
$\mathrm{id}(p)$	Unique participant identifier
σ	Cryptographic signature function
f_p	Participant state transition function
H	Cryptographic hash function
$C(p)$	Context Superstate of participant p
$\mathrm{Trust}(p)$	Trust sub-context; structural anchor of $C(p)$
$\mathrm{Assets}(p)$	Linear value objects
$\mathrm{Storage}(p)$	Participant-owned persistent data
$\mathrm{Logic}(p)$	Participant-owned computation
$\mathrm{Keys}(p)$	Cryptographic material
$\mathrm{Prefs}(p)$	Operational policies
$\mathrm{Intel}(p)$	EPI substrate; accumulated intelligence
$\alpha(h)$	Context chain anchor at height h
h	Context chain height
ι	Interaction 5-tuple $(S, T, W, f_\iota, \mathrm{Auth}_\iota)$
S	Source participant set
T	Target participant set
W	Witness set
f_ι	Interaction transformation function
Auth_ι	Cryptographic authorisations from source participants
$\mathrm{ICS}(\iota)$	Interaction Context Space (P_ι, C_ι, Φ)
P_ι	Participant set $S \cup T$
C_ι	Composite context $\bigoplus_{p \in P_\iota} C(p)$
Φ	Witness requirement policy
δ	ISM state transition function
$\sigma' = \Upsilon(\sigma, T)$	Classical global state transition (Cartesian model)
$\mathcal{T}(\iota)$	Tesseract for interaction ι
Π	Commit proof (IFP: $2f+1$ attestations)
f	Maximum Byzantine constituents
Δ	Network delay bound
R	Slot randomness
S_ι	Certified conflict set for interaction ι
cur_v	Current vote of constituent v
$\mathrm{Pos}(X)$	Support count for interaction X
$\mathrm{Neg}(Y)$	Rejection count for interaction Y
Π_e^v	Context Power of participant v in epoch e
E_e^v	Economic score
B_e^v	Behavioral score
S_e^v	Social score
ω_B, ω_S	Decay window parameters for B and S
$\Pi_{\min}$	Minimum Context Power threshold for witness eligibility

Symbol	**Meaning**
ι_e^v	Identity sub-component of Social score
ρ_e^v	Geo-distribution sub-component of Social score
G^v	Multiplicative gating function
$\mathrm{EPI}(P, t)$	Emergent Participant Intelligence; $= g_p(t)$ from CC3
$g_p(t)$	Metric tensor of participant p at time t
τ_p	KCC trajectory of participant p
$\mathcal{V}_{\mathrm{PIC}}$	PIC value category
$W(p)$	Witness set embedded in $\mathrm{Trust}(p)$

B CC Protocol Compliance Checklist (Normative)

This checklist is normative. A CC-compliant implementation must satisfy every item marked **MUST**. Items marked **MAY** are implementing network choices.

Context Protocol

MUST Single context vector per participant at any height. No branching, no forking.

MUST Context chain $\alpha(h{+}1) = H(\mathrm{id}(p)\|h{+}1\|\alpha(h)\|\sigma(h{+}1))$ maintained for every participant.

MUST Witness set $W(p)$ embedded in $\mathrm{Trust}(p)$ and committed in the context chain.

MUST No context transition without cryptographic authorisation from the context owner.

MUST Value is participant-indexed: Assets, Storage, Keys, and Intel exist only within the $C(p)$ of the participant they belong to. An inherited context C_i is itself a participant with its own sovereign $C(C_i)$—it may hold its own sovereign Assets, Storage, Logic, Keys, Prefs, and Intel. It may not hold value-bearing state on behalf of other participants.

MUST Context reset mechanism exists. Participant frozen at current valid height from moment of reset initiation.

MAY Specific context storage data structure (MDAG or other).

MAY Witness selection mechanism beyond the bound witness set.

MAY Specific hash function H.

Interaction State Machine

MUST State transition function scoped to named participant contexts only: $\delta(C(p_1), \ldots, C(p_n), \iota) = (C'(p_1), \ldots, C'(p_n))$.

MUST Two-bucket model enforced: value bucket linear, information bucket Cartesian, categorically distinct.

MUST Domain model includes participant, asset, and logic domains.

MUST Extension mechanism for additional interaction domains.

MUST Re-entrancy non-expressible at the type level, not merely detected at runtime.

MUST Atomicity, linearity, locality, and determinism hold for every execution.

MAY Programming language and execution environment (COCO or other).

MAY Specific asset standard implementation.

Interaction Finality Protocol

MUST Complete ICS fully determined before execution begins.

MUST Preparation threshold $2f+1$ of $3f+1$.

MUST Conflict resolution priority function deterministic and unpredictable in advance.

MUST Commit rule: $\text{Pos}(X) \geq 2f+1$ AND $\forall Y \neq X : \text{Neg}(Y) \geq f+1$.

MUST Confirmation state (cur_v, Pos, Neg, status) persists across views. Never reset by view timeout.

MUST Tesseract appears in the context chain of every participant named in P_ι, or in none. Partial commitment across named participants is not permitted.

MUST Tesseract embeds all four K-Machine dimensions: WHAT, WHERE, HOW, WHO.

MUST Pre-contexts consumed by a committed interaction not referenceable by any subsequent interaction.

MUST Safety and liveness under partial synchrony with at most f Byzantine constituents in a set of $3f+1$.

MAY Coordination mechanism (operator-assisted or fully leaderless), provided safety and liveness are preserved.

MAY Slot randomness derivation mechanism.

MAY Timing parameters: slot duration, view length, timeout thresholds.

MAY Specific f per interaction (governed by Φ).

Context Power

MUST Three-component structure: Economic (E), Behavioral (B), Social (S).

MUST Behavioral and Social scores non-transferable.

MUST Decay mechanism for B and S: bounded memory, past contributions eventually zero.

MUST Dynamic Economic cap: decreases as B and S accumulate.

MUST Bootstrap mechanism: Economic weight dominant at genesis, declines as network matures.

MUST Minimum Context Power threshold $\Pi_{\min}$ for witness eligibility is multi-dimensional.

MUST Multiplicative gating function G^v ensures all three dimensions must be present.

MAY Specific epoch duration.

MAY Decay window parameters ω_B, ω_S.

MAY Reward rate formulas.

MAY Component weight evolution schedule.

MAY Identity provider governance procedures.

MAY Geo-distribution scoring parameters.

MAY Token structures (bMOI/sMOI or equivalent).

C Derivation Map

This appendix presents the complete derivation chain from CC2–CC4 through CC5 normative requirements. For each requirement, the immediate source and the root axiom or theorem are given.

CC5 Requirement	**Immediate Source**	**Root (CC2/CC3/CC4)**
Participant sovereignty	CC4 Axiom 7.2 (Consent)	CC3: participant as manifold point; identity is irreducible
Value linearity	CC4 Axiom 7.3 (Linearity)	CC2: Domain Separation Theorem; $\mathcal{V}_{\mathrm{PIC}}$ has no diagonal
Context chain structure	CC4 Section 5.6	CC3: KCC trajectory τ_p; irreversible, monotone
Shared compute, no shared state	CC4 Axiom 7.6 (Commutativity)	CC2: value is participant-indexed; anonymous containers are absent
ICS pre-determination	CC4 Definition 5.7; Section 5.13	CC3: ICS is constituted by named participants, not discovered
Two-bucket ISM model	CC4 domain separation consequences	CC2: Cartesian Degeneracy—value and information are categorically distinct
$2f+1$ of $3f+1$ threshold	CC4 Axiom 7.4 (Witness Sufficiency)	CC3: KCR witness set within geodesic radius ε
Confirmation state view-persistent	CC4 Theorem 5.11 (Value–Agreement Equivalence)	Agreement is constituted, not discovered; cannot be undone
Tesseract atomicity	CC4 Axiom 7.3; Corollary 5.12	CC3: context singularity f_{pq} is joint; bilateral or absent
Re-entrancy non-expressible	CC4 Axiom 7.3 (Linearity)	Pre-state consumed at execution start; no reference survives
Witness embeddedness in Trust(p)	CC4 Section 5.22	CC3: witnesses are constituents, not external observers

CC5 Requirement	**Immediate Source**	**Root**
CP non-transferability	CC3 EPI non-transferability	Metric tensor $g_p(t)$ is participant-specific; no isometry transfers it
CP decay	CC3 $g_p(t)$ evolution	Geodesics not traversed lengthen; geometric form of forgetting
CP multi-dimensionality	CC3 Proposition 7.1	Security from participation, not capital; three irreducible dimensions

References

[1] A. Krishnan, "Contextual Compute—I: Philosophical and Quantum-Theoretic Foundations," Working Paper CC1, Sarva Labs, Princeton, NJ, 2025.

[2] A. Krishnan, "On the Cartesian Degeneracy of Distributed Value Systems," Zenodo, 2026. DOI: 10.5281/zenodo.19148907.

[3] A. Krishnan, "A General Theory of Contextual Compute: The K-Machine, Participant Manifolds, and the Recovery Theorem," Zenodo, 2026. DOI: 10.5281/zenodo.19177337.

[4] A. Krishnan, "PIC Semantics: Participant-Interaction-Context as the Operational Model of the K-Machine," Zenodo, 2026. DOI: 10.5281/zenodo.19189687.

[5] L. Lamport, "Time, Clocks, and the Ordering of Events in a Distributed System," *Communications of the ACM*, vol. 21, no. 7, pp. 558–565, 1978.

[6] C. Dwork, N. Lynch, and L. Stockmeyer, "Consensus in the Presence of Partial Synchrony," *Journal of the ACM*, vol. 35, no. 2, pp. 288–323, 1988.

[7] M. Castro and B. Liskov, "Practical Byzantine Fault Tolerance," in *Proc. 3rd USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pp. 173–186, 1999.

[8] M. Yin, D. Malkhi, M. K. Reiter, G. G. Gueta, and I. Abraham, "HotStuff: BFT Consensus with Linearity and Responsiveness," in *Proc. 38th ACM Symposium on Principles of Distributed Computing (PODC)*, pp. 347–356, 2019.

[9] L. Lamport, "The Part-Time Parliament," *ACM Transactions on Computer Systems*, vol. 16, no. 2, pp. 133–169, 1998.

[10] S. Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System," 2008. `https://bitcoin.org/bitcoin.pdf`.

[11] J.-Y. Girard, "Linear Logic," *Theoretical Computer Science*, vol. 50, no. 1, pp. 1–101, 1987.

[12] S. Abramsky, "No-Cloning in Categorical Quantum Mechanics," in *Semantics and Logic of Computation*, Cambridge University Press, pp. 1–28, 2009.

[13] B. Coecke and A. Kissinger, *Picturing Quantum Processes: A First Course in Quantum Theory and Diagrammatic Reasoning*, Cambridge University Press, 2017.

[14] T. Fox, "Coalgebras and Cartesian Categories," *Communications in Algebra*, vol. 4, no. 7, pp. 665–667, 1976.

[15] S. Micali, S. Vadhan, and M. Rabin, "Verifiable Random Functions," in *Proc. 40th Annual Symposium on Foundations of Computer Science (FOCS)*, pp. 120–130, 1999.

[16] C. Shannon, "A Mathematical Theory of Communication," *Bell System Technical Journal*, vol. 27, pp. 379–423, 1948.

[17] S. Abramsky and B. Coecke, "A Categorical Semantics of Quantum Protocols," in *Proc. 19th Annual IEEE Symposium on Logic in Computer Science (LICS)*, pp. 415–425, 2004.

[18] P. Wadler, "Linear Types Can Change the World," in *Programming Concepts and Methods*, North Holland, 1990.

[19] L. Lamport, R. Shostak, and M. Pease, "The Byzantine Generals Problem," *ACM Transactions on Programming Languages and Systems*, vol. 4, no. 3, pp. 382–401, 1982.

[20] J. Kwon, "Tendermint: Consensus without Mining," Technical Report, 2014. `https://tendermint.com/static/docs/tendermint.pdf`.

[21] V. Buterin, "Ethereum: A Next-Generation Smart Contract and Decentralised Application Platform," 2014. `https://ethereum.org/en/whitepaper/`.

[22] V. Buterin and V. Griffith, "Casper the Friendly Finality Gadget," arXiv:1710.09437, 2017.

[23] S. Blackshear, E. Cheng, D. L. Dill, V. Gao, B. Maurer, T. Nowacki, A. Pott, S. Qadeer, D. Rain, S. Sezer, T. Soares, K. Wu, and Y. Yan, "Move: A Language with Programmable Resources," Libra Association Technical Report, 2019.

[24] M. Sporny, D. Longley, M. Sabadello, D. Reed, O. Steele, and C. Allen, "Decentralised Identifiers (DIDs) v1.0," W3C Recommendation, 2022. `https://www.w3.org/TR/did-core/`.

[25] M. Sporny, D. Longley, and D. Chadwick, "Verifiable Credentials Data Model v1.1," W3C Recommendation, 2022. `https://www.w3.org/TR/vc-data-model/`.